

Enhancing Childbirth Simulation Using Unreal Engine's Rendering Pipeline

Rijo Thomas

Dissertation report submitted for the MSc Dissertation Module,
School of Computing Sciences, University of East Anglia, 2024/25

Abstract

This dissertation presents a real-time simulator of fetal head passage through the maternal pelvis, implemented in Unreal Engine using a deformable mass–spring sphere interacting with a toroidal birth canal. The fetal head is represented as a UV-sphere of structural, shear and bending springs, advanced with semi-implicit Euler integration, global damping and a gentle rest-shape pull for stability and recovery. A Jacobi-style distance-constraint projection curbs overstretch and suppresses axial wiggle. The pelvis is modeled as a procedural torus with adjustable radii, supporting analytic collision in which penetrating nodes are projected outward and inward normal velocities clamped to prevent chatter. A balloon-like squeeze-and-advance motion is created through the outlet as a result of user input building up as a decaying "click-to-pressure" charge that is transformed into localized force fields at the rim. Procedural mesh generation rebuilds the deformed surface each frame, while a Slate UI exposes parameters such as sphere radius, stiffness, damping, dilation and opposing force for interactive tuning. Stability at real-time frame rates is guaranteed by substepping. The framework shows how biomechanical processes can be interactively visualized by fusing physically based deformation, analytic collision and custom interaction design. This serves as a basis for future extensions to anatomical meshes and higher-fidelity tissue models, as well as educational applications.

Acknowledgement

I would like to express my heartfelt gratitude to my supervisor, Professor Rudy Lapeer, for his continuous support, insightful feedback and encouragement throughout the development of this dissertation. His expertise and guidance were invaluable at every stage of this research.

I am also thankful to Professor Vincent Moulton, who served as the module organizer for this dissertation. His structured approach and clarity in outlining expectations greatly contributed to the successful completion of this work.

Special thanks to the faculty and staff of the Department of Computing Science for fostering a supportive academic environment. I also appreciate my peers and friends for their constructive discussions and moral support along the way. My grateful thanks are also extended to Jaison Sebastian and Rajeev Muraleedharan, my managers at UST who were of great support at times when work deadlines conflict with my studies.

Nobody has been more important to me in the pursuit of this project than the members of my family. I wish to thank my loving and supportive wife, Elizabeth Janice and my wonderful daughter, Nila for her cute gestures. I would like to thank my mother, my brothers, my parents-in-law, brother-in-law and his wife, and Mother Sicily for her prayers.

Contents

1	Introduction	7
1.1	Background	7
1.1.1	Limited Visualization	8
1.1.2	Generic Anatomical Models	8
1.2	Motivation	9
1.3	Problem Scope	9
1.3.1	Goals	10
1.3.2	Objectives	10
2	Literature Review	11
2.1	Childbirth Simulation	11
2.2	Rendering in Medical Simulation	12
2.3	Mass–Spring Model (MSM) Foundations	12
2.4	Constraint Projection and Position-Based Ideas	13
2.5	Collision Handling as Projection	13
2.6	Volume Preservation for Surface Models	13
2.7	Time Integration and Damping	14
2.8	Procedural Meshes in Unreal	14
2.9	Existing Tools for Geometry Preparation and Runtime Import	14
2.10	Medical Simulation Context	15
3	Methodology	15
3.1	Research Objectives and Questions	15
3.2	Mass–Spring Network Construction	16
3.3	Force Computation	16
3.4	Time Integration and Stability	17
3.5	Distance Constraint Projection	18
3.6	Gentle Rest–Shape Pull	19
3.7	Collision Detection and Response	19

3.8	External Forces and User Interaction	20
3.9	Torus Collision	21
3.10	Solver Outline	22
3.11	Tools, Frameworks, and Implementation	23
3.12	Variables and Metrics	23
3.13	Experimental Protocol	23
3.14	Success Criteria	24
4	System Design	25
4.1	Overall Architecture	25
4.2	Per-Frame Execution Flow	25
4.3	Data Structures	27
4.4	Simulation Loop	27
4.5	Mesh Topology and Rendering	28
4.6	User Interface and Interaction Design	29
4.7	Collision Algorithm Integration	29
4.8	Integration with Anatomical Meshes	30
5	System Implementation	30
5.1	Code Map	30
5.2	Spherical Node Placement & Mesh Rebuild	31
5.3	Defaults and Tuning	32
5.4	Performance	32
5.5	Spring Construction	33
5.6	Simulation Code Summary	34
5.7	Procedural Mesh Update	34
5.8	UI Controls	35
5.9	Collision and Deformation	36
5.10	User Interaction	37
5.11	Actor Movement and Collision	38
5.12	Pelvis Mesh Import	38
5.13	Cervix Dilation Control	40

5.14	Performance Considerations, Build Configuration & Optimizations	40
6	Experiments and User Evaluation	41
6.1	Protocol	41
6.2	Measures	41
6.3	Ablations (Qualitative)	41
6.4	Results	41
6.5	Takeaways	44
7	Limitations and Future Work	44
8	Conclusion	45

1 Introduction

The biomechanics of childbirth is a complex process which involves interactivity of the fetal head and maternal pelvic structures. As these interactions and variations in anatomy can result in morbidity for both the mother and fetus, it is imperative to understand and predict these outcomes for effective obstetric training and efficient clinical decision-making. Rather than replacing the 3D simulation by [Lapeer et al., 2019], this project explores how Unreal Engine can be used to augment the BirthView Simulator by improving its user interface (UI) and delivering more immersive and visually compelling simulations using a sphere and torus mesh representing fetus head and maternal pelvis respectively.

1.1 Background

Childbirth is a complex biomechanical process which involves the interactivity of the fetal head with maternal pelvic anatomy in which both are propelled by uterine contractions and voluntary pushing. These interactions cause a sequence of cardinal movements (CMs) which are known as flexion, internal rotation, extension and external rotation that aid the fetus in moving through the birth canal. However, variations in maternal pelvic geometry, fetal size or positioning comes with complications such as shoulder dystocia, obstructed labor or pelvic floor trauma. Maternal complications are conceptually represented in Figure 1. [Vogel et al., 2024]. Therefore, it is critical to address the need for advanced training and predictive tools in obstetrics in order to mitigate these risks and complications.

The biomechanics of childbirth can now be effectively modeled using computational simulations. Early efforts, such as [McPherson and Kriewall, 1980], used finite element analysis (FEA) to study fetal cranial deformation, but were limited by computational power. Evidently, the advancements in real-time simulation paved the way to the creation of BirthView by [Lapeer et al., 2019], a VR simulator that uses Total Lagrangian Explicit Dynamics (TLED) for soft tissue deformation and partial Dirichlet-Neumann (pDN) methods for simulating fetal-maternal interactions. BirthView delivers real-time performance (≤ 16 ms per frame) running on a GPU, accurately simulating CMs by including soft tissues such as the pelvic floor muscles, uterine cervix and sacrospinous ligaments. However, despite its technical successes, BirthView has some notable limitations.

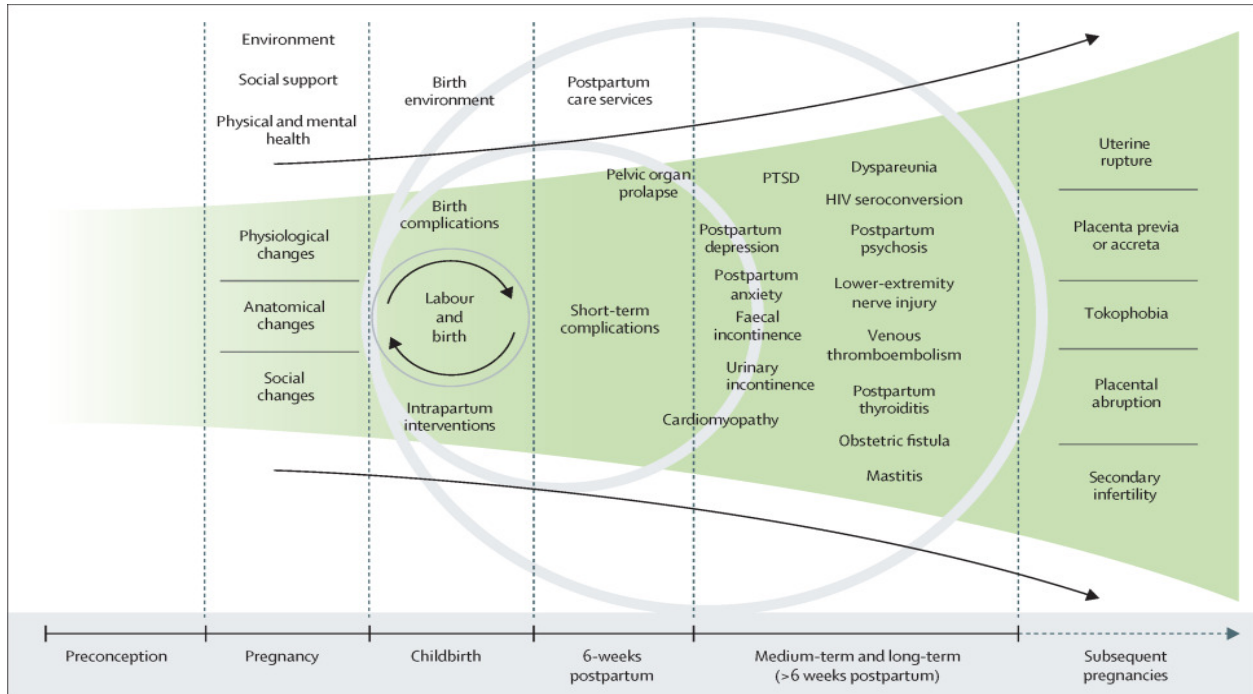


Figure 1: Representation of different conditions that arise from labour and childbirth. Source [Vogel et al., 2024]

1.1.1 Limited Visualization

BirthView simulator's rendering capabilities are basic and presents simple graphics that fall short of the photorealism and interactivity which are essential for immersive training. Modern game engines such as Unreal Engine offer advanced rendering features (such as real-time ray tracing, dynamic lighting and material systems) that could greatly improve the simulator's educational value.

1.1.2 Generic Anatomical Models

The ability of the current BirthView system to replicate patient-specific conditions is limited because it mainly uses generic, average anatomical representations. This caveat can be resolved by extending the platform to add a feature which allows loading and switching of various anatomical models straight into the simulation such as custom pelvis meshes or other anatomical structures. Because of its adaptability, the simulator can be customized for a variety of situations like training or research use cases that are unique to a patient.

1.2 Motivation

Real time obstetric training benefits from deformable models that are responsive, stable and easy to tune. High order continuum models like FEM, [Lapeer and Prager, 2001] can be accurate but are heavyweight for interactive use. A surface mass–spring model with constraint projection offers a practical balance between plausibility and performance [Va et al., 2023]. Many existing solutions depend on high-level physics engines, but this project builds a custom soft-body model for better control and understanding.

In contemporary computer graphics, virtual reality and interactive simulations, the creation of realistic physically based simulations has become increasingly important. Soft body physics is essential to realistic simulation in many fields, such as medical education, game development, robotics and engineering visualization. Realistic soft-body deformation in real time is still a major challenge, though. This project report describes the design and implementation of a deformable sphere simulation that interacts with a toroidal (ring-shaped) structure using a mass-spring system in Unreal Engine. The compliance and squishiness of materials such as rubber or biological tissue are not captured by conventional rigid-body engines. On a typical PC/modern hardware, mass-spring networks offer a simple, small and easily adjustable approximation that can operate at interactive rates. The system simulates a deformable sphere in real time, with visual reactivity and physical realism, by utilizing procedural mesh generation and custom physics computation. The project shows how spherical geometries can be dynamically deformed by integrating traditional mass-spring modeling principles with the Unreal Engine rendering pipeline. Real-time deformation visualization, force adjustments and interactive sphere parameter modification are all available to users.

1.3 Problem Scope

Existing game engines offer built-in cloth and soft-body systems, but these are often tied to proprietary collision and performance heuristics [Zhang, 2024]. My goal was to implement, from first principles in C++, a fully custom deformable sphere that can be squeezed through a rigid torus without using Unreal’s proprietary soft-body or collision subsystems. This project develops SphereDeformActor which represents the fetal head, a custom C++ Unreal Engine actor that employs a mass–spring system for soft-body deformation, integrates a collision detection algorithm

against a TunnelTorusActor (a procedurally generated toroidal tunnel) which represents the pelvic mesh imported dynamically via C++ procedural mesh handling. Unlike static mesh import workflows, the system loads an external torus object that represents a pelvic model file (OBJ or OFF) at runtime, ensuring flexibility for different anatomical geometries and enabling research workflows using academic datasets. The framework also provides a Slate-based user interface for live parameter tuning. Traditional simulators rely heavily on predefined animations or commercial physics engines, which limit flexibility and increase cost.

1.3.1 Goals

The goal is twofold:

- **Educational Utility:** Offer trainees an interactive, deformable sphere whose behavior under collision mimics tissue response.
- **Research Platform:** Provide a modular framework for extending to patient-specific models (e.g., fetal head and maternal pelvis) using the Unreal rendering pipeline, custom physics, and collision logic, without relying on proprietary engine physics.

1.3.2 Objectives

This project creates an analytical collision detection framework for sphere–torus interactions and uses a custom mass–spring system to implement a deformable sphere. The sphere can deform enough to fit through the torus hole, thanks to the integration of dynamic deformation under external forces and mouse-based deformation. Without depending on the game engine’s integrated collision handling, a UV-sphere mass–spring network is built to support user-driven pokes and analytic torus collisions, allowing for visible and localized squishiness at contact points while maintaining the overall shape and avoiding interpenetration. The system is intended to function as a low-level, real-time fetal head–pelvis simulation while maintaining interactive frame rates above 60 frames per second. This is accomplished by purposefully eschewing Unreal Engine’s proprietary skeletal and physics pipelines in favor of procedural mesh rendering and custom physics. The framework offers tools for dynamically modifying pelvic canal size, fetal head stiffness and dilation, and it supports the dynamic import of external pelvic meshes (OBJ/OFF) without depending on the Unreal Editor asset import pipeline. Finally, the system’s performance and usability in

research and medical education applications are assessed.

2 Literature Review

2.1 Childbirth Simulation

Earlier models studied by [McPherson and Kriewall, 1980] developed a finite element model using SAP IV to quantify fetal parietal bone molding during labor which predicts diametral strains and preterm deformations. The linear, single-bone model, lacking sutures and brain provides a foundational but simplified benchmark for modern childbirth simulations, however, limited by computational constraints. [Lapeer and Prager, 2001] developed a non-linear finite element model to simulate fetal skull molding during labor using a ABAQUS FE software, capturing skull deformation under cervical pressures. Although, it incorporates sutures and fontanelles and has clinical validation, it is limited by static assumptions and a single geometry unlike the more dynamic simulators used today. In a subsequent paper, [Lapeer et al., 2004] describe a system designed to simulate forceps delivery using augmented reality (AR). The incorporation of AR enhances the simulation's appeal which has more usability in training applications. However, the simulation's main limitation is its inability to include the real-time deformation of the fetal head in contact with the forceps. [Lien et al., 2004] constructed a 3D structural model to simulate levator ani muscle stretch during vaginal birth using MRI images and its effect on the pelvic floor muscles. However, manually created pelvic floor structures doesn't fit in for a patient-specific solution. The study conducted by [Ashton-Miller and DeLancey, 2009] reveals vaginal birth induces extreme pelvic floor stretches which causes levator ani injuries in 20% of primiparous women, linked to pelvic organ prolapse via MRI-validated finite element models. The study highlights forceps-related risks and pregnancy's tissue adaptations but is limited by simplified geometry models and static loading conditions unlike dynamic patient-specific simulators. In the paper [Gerikhanov et al., 2013], they created a VR childbirth simulator that models descent, engagement and flexion using rigid fetal skull and pelvis models. However, it requires waypoints for later cardinal movements because it lacks soft tissues and articulated fetus. [Lapeer et al., 2019], in BirthView simulator addressed this by incorporating pelvic floor muscles, uterine cervix and ligaments with the help of TLED for deformation and pDN for contact and achieved real-time CM replication. Even though, BirthView is a remarkable solution in offering valuable insights, its basic rendering limits its educational

potential. Additionally, the constraints of using average anatomical models restrict the ability to create patient-specific simulations, consequently, reducing its applicability in personalized medical training.

2.2 Rendering in Medical Simulation

[Eagleson et al., 2024]’s study from 2024 demonstrates Unreal Engine’s potential for medical visualization by creating realistic XR neurosurgical simulations that offer real-time 3D visualization of patient-specific neuroanatomy to improve spatial reasoning and training. While [Pires et al., 2021] explore Unreal Engine’s integration with VTK for immersive VR medical imaging visualization with improvement on 3D analysis of DICOM data for clinical applications, [El-Wajeh et al., 2022] show how to improve procedural accuracy and patient outcomes by utilizing Unreal Engine 5’s sophisticated features, such as Nanite and Lumen to create immersive XR surgical training simulations. Unreal’s Rendering Hardware Interface (RHI) allows for low-level geometry submission, as shown by custom visualization frameworks [Epic Games, 2022]. A careful decoupling is necessary to avoid reliance on proprietary systems like PhysX because the rendering in Unreal Engine and the physics engine in BirthView have not yet been integrated.

2.3 Mass–Spring Model (MSM) Foundations

Mass–spring surfaces place point masses at mesh vertices and connect them with springs tuned to distinct deformation modes, specifically, *structural* (edge stretch), *shear* (in–plane skew) and *bending* (out–of–plane curvature) to emulate thin–shell behaviour with local computations. This canonical triplet goes back to early cloth work, which enumerates the spring types and derives internal forces from Hooke-like terms [Provot, 1995]. For real-time soft bodies, MSMs are widely adopted as a pragmatic alternative to FEM. They map elasticity to springs between nodes and are easy to tune for stability/performance trade-offs. To address the main pain points such as stability at large time steps and robust constraints/collisions, the literature introduced large-step implicit integration for cloth [Baraff and Witkin, 1998] and later energy-minimization/local–global solvers for stiff systems [Bouaziz et al., 2014]. Surveys position MSMs among the principal families (MSM, FEM, PBD/XPBD) for graphics deformables [Nealen et al., 2006].

2.4 Constraint Projection and Position-Based Ideas

Explicit integration of stiff springs is step-size sensitive. A practical workaround is to treat rest distances as *constraints* and *project* vertex positions toward them each substep (Jacobi or Gauss–Seidel style). Game-oriented practice popularized this approach [Jakobsen, 2001] and it was later formalized as Position-Based Dynamics (PBD) [Müller et al., 2007]. Extended PBD (XPBD) introduces *compliance* so that perceived stiffness becomes less dependent on the time step and iteration count, while also yielding meaningful constraint forces [Macklin et al., 2016]. In my setting, a small number of projection passes per substep emulates high stiffness without an implicit solve, making PBD/XPBD a natural fit for interactive MSMs. Related projection-style solvers, such as Projective Dynamics, fuse constraints into a fast global/local optimizer with strong convergence [Bouaziz et al., 2014].

2.5 Collision Handling as Projection

A robust real-time pattern is “collision as projection” which detects penetrations, push points to the admissible side along contact normals and damp inward normal velocity to reduce chatter. This pairs well with PBD/XPBD loops and classic cloth pipelines [Bridson et al., 2002, Teschner et al., 2005]. When analytic signed-distance queries are available, (for instance, sphere–torus), per-vertex penalty or direct projections are cheap and stable. For general convex shapes, the narrow phase commonly uses GJK for closest-feature queries [Gilbert et al., 1988] and EPA for penetration depth and contact normals. The GJK/EPA combination underpins many production engines like Unreal Engine. Practical options therefore include: (i) analytic collision for simple primitives (sphere–sphere, sphere–torus), (ii) convex mesh–mesh via GJK/EPA, and (iii) broad-phase culling (grids/BVHs) followed by per-vertex projections or temporary “virtual springs” [Bridson et al., 2002, Teschner et al., 2005].

2.6 Volume Preservation for Surface Models

Surface-only MSMs tend to lose volume under compression. Two standard remedies are: (i) switch to volumetric elements with per-element volume constraints (heavier) or (ii) impose a global surface-based volume constraint that computes enclosed volume and applies corrective position/force updates. The latter is simple and performant for real time and has been demonstrated

effectively for surface MSMs with GPU implementations [Va et al., 2023].

2.7 Time Integration and Damping

A lightweight stepper combines semi-implicit (symplectic) Euler with velocity damping, stabilized by constraint projection, a graphics tradition since [Terzopoulos et al., 1987]. If fewer substeps with stiffer behaviour are required, implicit or predictor–corrector schemes for structured deformables are canonical alternatives [Baraff and Witkin, 1998, Desbrun et al., 1999].

Shape memory / drift control. A small goal-pull toward rest shape per substep, a low-gain cousin of shape matching that arrests drift and heals dents and integrates neatly with PBD/XPBD [Müller et al., 2005].

External forces (localized with falloff). Injecting forces to nodes inside a contact radius around tools/handles is standard in force-driven MSMs before the projection stage corrects violations [Müller et al., 2007, Macklin et al., 2016].

Substepping and iterations. A $\text{SubSteps} \times \text{ConstraintIters}$ schedule aligns with PBD practice; XPBD’s compliance then de-sensitizes apparent stiffness to Δt and iteration count [Macklin et al., 2016].

2.8 Procedural Meshes in Unreal

Unreal’s `UProceduralMeshComponent` enables runtime mesh construction from vertex/index arrays, ideal for programmatic spheres/tori and for bypassing the editor’s asset-import pipeline. OBJ/OFF can be loaded at runtime via lightweight importers (e.g., Assimp), preserving full control over custom physics and collision. This deliberately avoids Chaos/PhysX heuristics when deterministic, reproducible research code paths are needed [EpicGames, 2025b, EpicGames, 2025a, Assimp, 2025].

2.9 Existing Tools for Geometry Preparation and Runtime Import

Blender provides effective mesh cleanup/decimation and watertightness workflows for anatomical models (*Clean Up*, *Decimate*) [Blender, 2025]. **Assimp** supports many interchange formats and post-processing options for runtime ingestion and simplification [Assimp, 2025].

2.10 Medical Simulation Context

For interactive training, stable “balloon-like” responses are often sufficient initially. Anisotropy (for example, cranial plates, sutures) can be layered later using direction-dependent springs and regional stiffness masks. FEM studies of childbirth mechanics motivate target behaviours and validation metrics [Lapeer and Prager, 2001], and remain the accuracy gold standard albeit at higher computational cost. Patient-specific geometry from MRI/CT is desirable but non-trivial; synthetic data and generative models offer complementary strategies for personalization and risk assessment [Goodfellow et al., 2014]. Recent comparative studies also discuss trade-offs between engine cloth and open GPU PBD pipelines for real-time applications.

3 Methodology

3.1 Research Objectives and Questions

Creating a real-time simulation framework to model the deformation of a soft spherical body interacting with a constrained toroidal canal is the aim of this study. In biomedical contexts, this corresponds to simulating the passage of a fetal head through the pelvic canal, requiring a balance between mechanical realism and computational efficiency. While finite element methods (FEM) provide accuracy, they are often unsuitable for interactive training where immediate response is essential [Nealen et al., 2006]. By contrast, mass-spring systems (MSMs) offer a lightweight model capable of soft-body behavior at interactive rates [Nealen et al., 2006]. Critically, I pursue an implementation that eschews Unreal Engine’s proprietary physics/soft-body models in favor of a fully custom C++ pipeline for transparency, control and reproducibility.

In this project, I investigate the following questions: (i) can a mass-spring model (MSM) augmented with distance-constraint projection reproduce smooth squeezing and when user-initiated pushing is present, advance through a toroidal constraint? (ii) what stiffness and damping ranges yield stable yet responsive behavior, avoiding jitter or collapse? (iii) how do structural, shear and bending springs individually and jointly influence rim compliance during torus contact? (iv) what is the contribution of a gentle rest-shape pull (goal pull) in preserving global structure while permitting local compliance [Müller et al., 2005]? (v) how might a surface-based volume constraint be incorporated in future work to enhance anatomical plausibility while retaining real-time perfor-

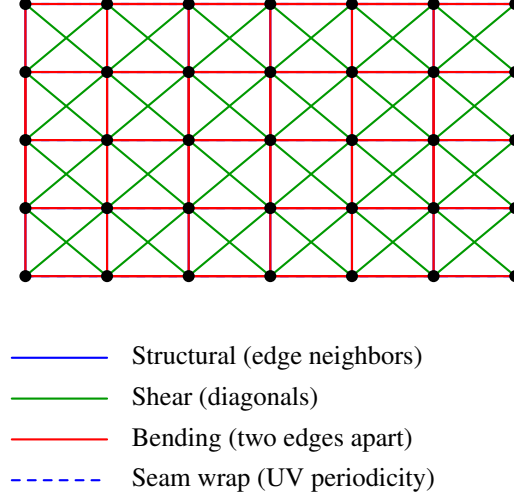


Figure 2: UV-sphere discretization with structural (blue), shear (green), and bending (red) springs.

mance [Va et al., 2023]? and (vi) can a fully custom, engine-native but physics-agnostic framework in Unreal (procedural meshes + bespoke collision/constraints) deliver interactive performance, reproducibility and extensibility to anatomical meshes without relying on proprietary models? These questions assess the feasibility of MSMs and a custom UE implementation as a basis for real-time medical training simulators.

3.2 Mass-Spring Network Construction

The deformable sphere is a UV-sphere lattice of mass nodes connected by springs. Each node stores position, velocity, a force accumulator and its rest (undeformed) position. The topology uses three spring families and this triplet follows common cloth MSM practice [Provot, 1995, Nealen et al., 2006]: (1) *structural* springs along latitude/longitude edges (resist stretch), (2) *shear* springs between diagonals (resist in-plane skew) and (3) *bending* springs two edges apart (resist out-of-plane curvature) as shown in Figure 2. Rest lengths are initialized at build time and remain fixed and spring stiffnesses are tunable parameters for study. This layered design balances local compliance and global stability.

3.3 Force Computation

At each substep, forces are accumulated from multiple sources. Elastic response is modeled with Hookean springs between node pairs A and B of the surface mesh. For a spring with rest length L_0

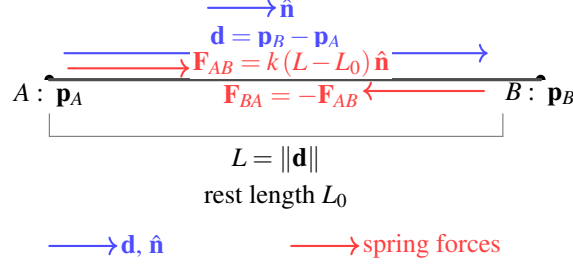


Figure 3: Spring between two nodes: elongation vector $\mathbf{d}$, unit direction $\hat{\mathbf{n}}$, and equal/opposite forces $\mathbf{F}_{AB}$ and $\mathbf{F}_{BA}$.

Table 1: Spring families and their primary roles in the MSM surface model.

Family	Connectivity	Primary effect
Structural	Immediate edge neighbors (latitude/longitude)	Resists edge stretch; establishes basic membrane tension and preserves local lengths.
Shear	Diagonals within each quad cell	Resists in-plane skew/shear distortion, maintaining element shape under contact.
Bending	Two edges apart (e.g., $(i+2, j)$ and $(i, j+2)$)	Resists curvature change; promotes smoothness and discourages sharp folds/creases.

and stiffness k , let $\mathbf{d} = \mathbf{p}_B - \mathbf{p}_A$, $L = \|\mathbf{d}\|$, and $\hat{\mathbf{n}} = \mathbf{d}/L$. The restoring forces are

$$\mathbf{F}_{AB} = k(L - L_0)\hat{\mathbf{n}}, \quad \mathbf{F}_{BA} = -\mathbf{F}_{AB},$$

which resist both stretch and compression. In implementation, springs are applied every sub-step and equal-and-opposite forces conserve linear momentum during force accumulation. After integration, a lightweight *constraint projection* pulls endpoints toward L_0 (splitting the correction 50/50) and removes relative velocity along each spring to suppress axial “wobble,” so length preservation is primarily enforced by projection rather than high stiffness alone. Typical coefficients are $k_{\text{struct}} \approx 8$, $k_{\text{shear}} \approx 6$ and $k_{\text{bend}} \approx 3$.

3.4 Time Integration and Stability

In this study, I adopt semi-implicit (symplectic) Euler integration for its simplicity and suitability for inter- active graphics [Terzopoulos et al., 1987].

$$\mathbf{v} \leftarrow (\mathbf{v} + \mathbf{F}\Delta t)\gamma, \quad \mathbf{x} \leftarrow \mathbf{x} + \mathbf{v}\Delta t,$$

where $\gamma \in (0, 1)$ is a global velocity damping factor. This scheme is lightweight and stable for the small substeps used here. In practice we set $\gamma \approx 0.98$ per substep to suppress ringing while preserving responsiveness. At each step, velocities are updated from forces (scaled by t and damping) before positions are advanced. The simulation employs sub-stepping, which divides the frame into smaller t updates and numerous constraint iterations per substep to enforce spring lengths and dampen axial wiggles in order to preserve stability under stiff springs and strong contact corrections. This is in line with standard procedure in real-time deformable simulation, where projection techniques stabilize explicit updates [Müller et al., 2007, Macklin et al., 2016]. Stability is evaluated empirically, i.e, displacements stay bounded, oscillations decay smoothly and the sphere gradually regains its rest shape when the right values of t , damping, substeps and iteration counts are chosen. More heavyweight alternatives such as implicit or predictor–corrector schemes exist [Baraff and Witkin, 1998, Desbrun et al., 1999], but they are not necessary for the present interactive rates.

3.5 Distance Constraint Projection

Integration alone permits small stretch errors to accumulate. We therefore perform a projection pass in the style of position-based dynamics (PBD): for each spring, compute the constraint violation $C = L - L_0$ and apply symmetric corrections

$$\Delta \mathbf{p}_A = +\frac{1}{2}C \hat{\mathbf{n}}, \quad \Delta \mathbf{p}_B = -\frac{1}{2}C \hat{\mathbf{n}}.$$

skipping fixed nodes. To reduce axial chatter, I also damp half of the relative velocity component along $\hat{\mathbf{n}}$. In practice, repeating this update for ~ 6 iterations per substep enforces near-rigidity without the cost of an implicit solver [Jakobsen, 2001, Müller et al., 2007].

While the implementation follows basic PBD, extended position-based dynamics (XPBD) [Macklin et al., 2016] generalizes this approach by introducing a *compliance* parameter that makes stiffness independent of time step and iteration count and exposes well-defined constraint forces.

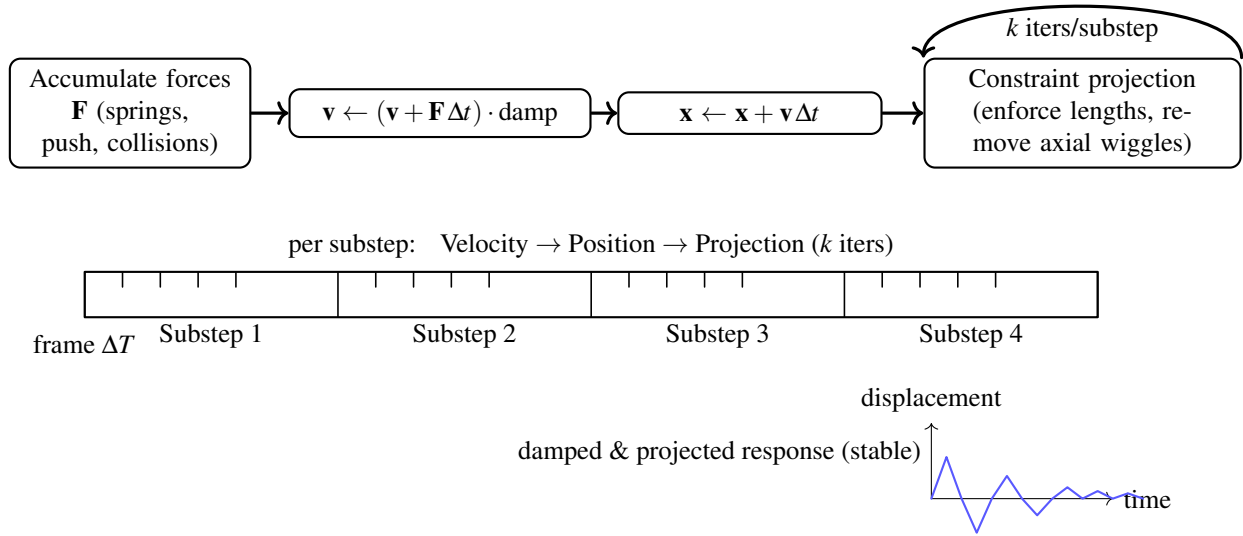


Figure 4: Semi-implicit (symplectic) Euler with sub-stepping and per-substep constraint iterations. Velocities update from forces before positions; constraints are projected multiple times to enforce spring lengths and suppress axial wiggles. With suitable Δt , damping, substeps and iterations, oscillations decay and displacements remain bounded.

3.6 Gentle Rest–Shape Pull

To combat slow drift and persistent dents, each node is weakly pulled toward its original rest position x^* :

$$x \leftarrow (1 - \alpha)x + \alpha x^*, \quad \alpha \ll 1 \text{ (e.g. } \alpha = 0.02\text{)}.$$

Equivalently, this may be viewed as a corrective displacement $\Delta x \leftarrow \Delta x + \alpha(x^* - x)$.

This update is a low-gain variant of the shape matching idea introduced by [Müller et al., 2005]. Applied after constraint projection at each substep, it gradually “heals” accumulated deformation and suppresses seam creep, while remaining weak enough not to erase legitimate flattening arising from sustained contact.

3.7 Collision Detection and Response

Collision detection is specialized to the torus geometry. Each node is transformed into torus local space, projected onto the ring plane and tested against the tube radius with a fit margin. If the node lies inside the tube, it is marked as colliding. This avoids generic GJK/EPA pipelines but achieves an analytic test tailored to the toroidal shape. Response follows a projection pattern: colliding

nodes are displaced outward along the local surface normal and inward normal velocity is clamped to zero to suppress chatter, consistent with standard cloth collision handling [Bridson et al., 2002]. When external pushing is applied, rim nodes deform locally and positional projection ensures the sphere can squeeze through the torus opening without numerical interpenetration.

3.8 External Forces and User Interaction

When the sphere is blocked and pressurized, a localized external field is applied to nodes within a contact radius R_c around the estimated contact anchor. Two contributions are used:

1. an inward *pinch* toward the sphere center (compression),
2. a directional *bulge* along the push axis, applied with opposite signs on the front and back sides.

Both are scaled by a smooth distance falloff. In my implementation this is a simple linear ramp

$$w(d) = \max\left(0, 1 - \frac{d}{R_c}\right),$$

where d is the distance to the contact anchor, though quadratic alternatives such as $1 - (d/R_c)^2$ are also common.

The effective push magnitude decays over successive frames (e.g. by multiplying by 0.9 each step), mimicking the fading of uterine contractions. External forces represent user-driven pushing. Mouse clicks accumulate a push charge, which applies directional forces to nodes within a world-space contact radius around the push origin. These forces combine an inward radial pinch with a slight forward bulge, both scaled by a radial falloff, so deformation is localized to the contact band. UI sliders (implemented via Slate) expose parameters such as sphere radius, damping, movement direction and opposing force, allowing interactive tuning while other parameters like spring stiffness are currently fixed in code but conceptually tunable. This setup mirrors Position-Based Dynamics workflows [Müller et al., 2007, Macklin et al., 2016], where user forces are injected first and then constraint projection restores admissibility and stability.

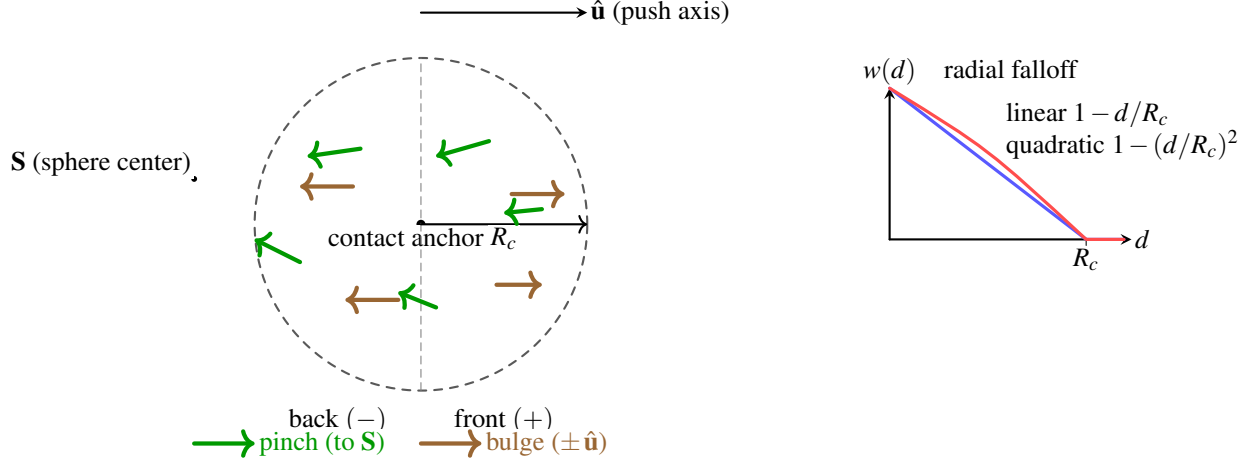


Figure 5: External field across the contact band: inward *pinch* toward $\mathbf{S}$ and directional *bulge* along $\pm \hat{\mathbf{u}}$, both scaled by a radial falloff $w(d)$ from the contact anchor. Front/back are split by the plane orthogonal to $\hat{\mathbf{u}}$ through the anchor.

3.9 Torus Collision

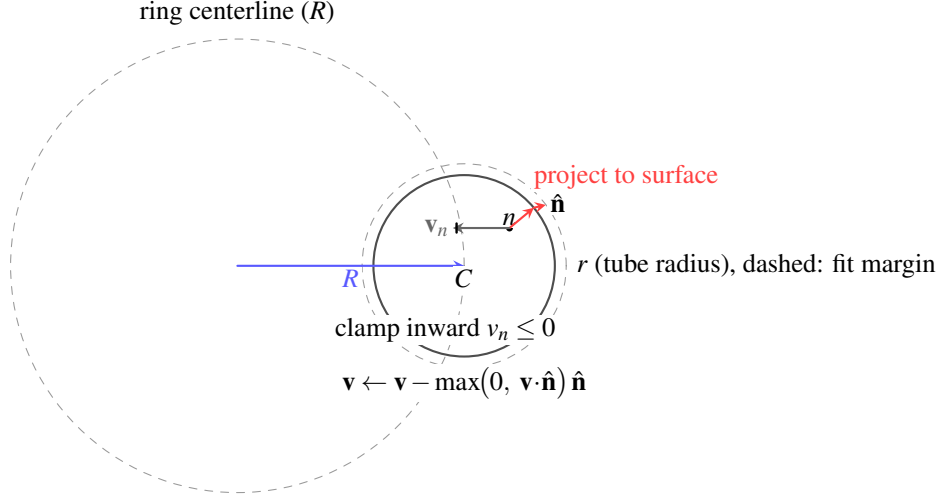
Broad-phase collision is detected using an analytic signed-distance formulation in the torus local coordinate system, with ring radius R and tube radius r . Nodes are flagged whenever their distance to the torus centerline is less than r plus a small fit margin.

For each penetrating node, the narrow-phase response is *collision as projection* [Bridson et al., 2002]: the node is displaced outward along the local surface normal to lie exactly on the boundary and its inward normal velocity component is zeroed to reduce chatter,

$$\mathbf{v} \leftarrow \mathbf{v} - \max(0, \mathbf{v} \cdot \hat{\mathbf{n}}) \hat{\mathbf{n}}.$$

This per-node projection is applied after spring integration and before mesh reconstruction, consistent with position-based dynamics pipelines.

Such geometric projection schemes are standard in deformable simulation, while more advanced approaches including continuous collision detection and robust response are reviewed in [Teschner et al., 2005].



Torus cross-section: projection and inward-velocity clamp

Figure 6: Torus collision (narrow phase) in a local tube cross-section. A penetrating node n is projected outward along the local normal $\hat{\mathbf{n}}$ to the surface and the inward normal component of velocity is clamped: $\mathbf{v} \leftarrow \mathbf{v} - \max(0, \mathbf{v} \cdot \hat{\mathbf{n}}) \hat{\mathbf{n}}$. Broad-phase uses the ring centerline with radius R ; the tube has radius r (dashed circle shows a small fit margin).

3.10 Solver Outline

Each simulation frame is subdivided into S substeps (typically $S = 12$) within each substep the solver executes:

1. Clear all node force accumulators.
2. Accumulate spring forces (structural, shear, bending) and associated velocity damping.
3. If blocked and pressurized, apply a localized external field (pinch + bulge) around the contact anchor.
4. Integrate with semi-implicit Euler and global damping factor γ .
5. Run I constraint projection iterations (typically $I = 6$), correcting distance errors and damping axial relative velocity along each spring. (Extended PBD with compliance [Macklin et al., 2016] could be used here, but is not enabled in this implementation.)
6. Apply a gentle rest-shape pull toward each node's rest position $\mathbf{x}^*$.
7. Resolve rigid torus contacts: project penetrating nodes out of the collider surface and zero inward normal velocity.

This ordering—*forces* $\rightarrow$ *integration* $\rightarrow$ *projection* $\rightarrow$ *collisions*—follows standard real-time deformable simulation practice [Müller et al., 2007, Macklin et al., 2016, Bridson et al., 2002]. Substepping together with a small number of projection iterations delivers smooth, responsive deformation at interactive rates.

3.11 Tools, Frameworks, and Implementation

The simulator is implemented in Unreal Engine 5 (C++). Both the deformable sphere and torus are rendered at runtime using UProceduralMeshComponent, bypassing Unreal’s Chaos physics to retain full control and reproducibility [EpicGames, 2025b, EpicGames, 2025a]. Anatomical meshes can be preprocessed in Blender (cleanup, decimation, scaling) and imported via a custom OBJ loader implemented in the code. This loader parses vertex and face data into Unreal’s procedural mesh system, ensuring direct runtime use. Support for additional formats (e.g., OFF, FBX, STL) could be added in future through libraries such as Assimp, which provides a unified interface for multiple 3D file formats [Blender, 2025, Assimp, 2025]. Runtime performance is tuned for interactive rates (60–120 FPS on desktop). The most demanding phases are spring force accumulation and procedural mesh rebuild. Currently, loops are serial, but the flat array-of-structs layout is cache-efficient and future work could parallelize these kernels to further improve throughput.

3.12 Variables and Metrics

Independent variables include spring stiffnesses ($k_{\text{struct}}, k_{\text{shear}}, k_{\text{bend}}$), damping, number of constraint iterations, substep count, contact radius, fit margin and applied push magnitude. Dependent variables comprise traversal time, number of clicks (user effort), minimum and average FPS, blocked time fraction (proportion of the trial spent obstructed), and subjective smoothness (Likert). Together these metrics characterize numerical stability, runtime performance, and perceptual realism of the simulation.

3.13 Experimental Protocol

In the experiment, I conduct parameter sweeps over spring stiffnesses, damping and push force, and perform ablation studies by selectively disabling shear springs, bending springs, rest-shape pull or constraint projection to isolate their contributions. Five participants complete trials first

Table 2: Independent and dependent variables used in the simulator. Unreal Engine uses centimeters for length and arbitrary units (au) for force/energy.

Type	Variable	Definition / Unit
Independent	$k_{\text{struct}}, k_{\text{shear}}, k_{\text{bend}}$	Spring stiffnesses [au]
Independent	Damping	Velocity damping coefficient [–]
Independent	ConstraintIters, SubSteps	Iterations per substep, substeps per frame [–]
Independent	Contact radius, Fit margin	Collision/contact thresholds [cm]
Independent	Push magnitude	External push amplitude (ClickPushCharge $\times$ multiplier) [au]
Dependent	Traversal time	Time to pass torus [s]
Dependent	Click count	User input effort [#]
Dependent	FPS stability	Min/avg FPS during trial [Hz]
Dependent	Blocked fraction	Proportion of trial time spent obstructed [%]
Dependent	Smoothness (Likert)	Subjective rating [1–5]

with default settings, then with varied parameters. Trials are run on a mid-range desktop PC equipped with an RTX 2060 GPU (8 GB VRAM) and 8 GB system RAM, under Unreal Engine 5.5. Quantitative metrics (traversal time, click count, FPS, blocked fraction) are logged during each trial and qualitative ratings (smoothness, realism) are collected afterward. This triangulated protocol allows evaluation of both technical stability and perceived plausibility of the simulation, even under constrained performance conditions (4–6 FPS on the present hardware).

3.14 Success Criteria

Success is defined by numerical stability (no divergence or runaway oscillations), minimal visible jitter, progression through the torus under repeated user push, smooth rim flattening at contact, and gradual elastic recovery upon release. Runtime performance in the present setup sustains only 4–6 FPS due to hardware and engine constraints, however, the system is designed to achieve interactive rates (≥ 60 FPS) on higher-end GPUs with more memory. Subjective participant ratings confirm that, despite low framerates, deformation felt smooth and plausible. Meeting these criteria demonstrates that a mass–spring model augmented with collision projection, a gentle goal pull and constraint enforcement can serve as a practical foundation for real-time childbirth simulation tools [Müller et al., 2007, Macklin et al., 2016, Bridson et al., 2002, Baraff and Witkin, 1998, Desbrun et al., 1999].

4 System Design

4.1 Overall Architecture

The system comprises three principal components for simulating a deformable (mass–spring) sphere advancing through a toroidal or pelvic canal.

1. **Sphere Control Actor** (`ASphereControlActor`) coordinates user engagement, pressure application scheduling of the simulation loop, forward translation, broad-/narrow-phase contact tests and replication. For instant feedback, it also binds runtime user interface controls such as sphere radius, movement direction, damping and opposing force to simulation parameters.
2. **Mass–Spring System** (`FMassSpringSystem`) encapsulates deformable dynamics: node/spring arrays, force accumulation, semi-implicit time integration, iterative constraint projection (PBD/XPBD-style) and a gentle goal-pull to curb drift [Müller et al., 2005, Müller et al., 2007, Macklin et al., 2016]. Physics is deliberately decoupled from rendering for clarity and testability.
3. **Tunnel Torus Actor** (`ATunnelTorusActor`) provides constraining geometry (procedural torus or imported anatomy), exposes collision checks via analytic torus geometry and allows ring/tube radii to be tuned at runtime. Procedural meshes for both sphere and torus are generated with Unreal’s `UProceduralMeshComponent` [EpicGames, 2025b, EpicGames, 2025a].

4.2 Per-Frame Execution Flow

For each frame (`Tick()`), the control actor performs the following

1. **Broad phase:** estimate proximity to the torus; compute a contact anchor on the ring and mark in-band contact.
2. **Narrow phase:** if in-band, test sphere nodes against the torus tube; flag *blocked* if any node penetrates.
3. **External push:** if blocked and click charge > 0 , set push direction/magnitude, else no push.
4. **Substep loop** ($s = 1..S$):

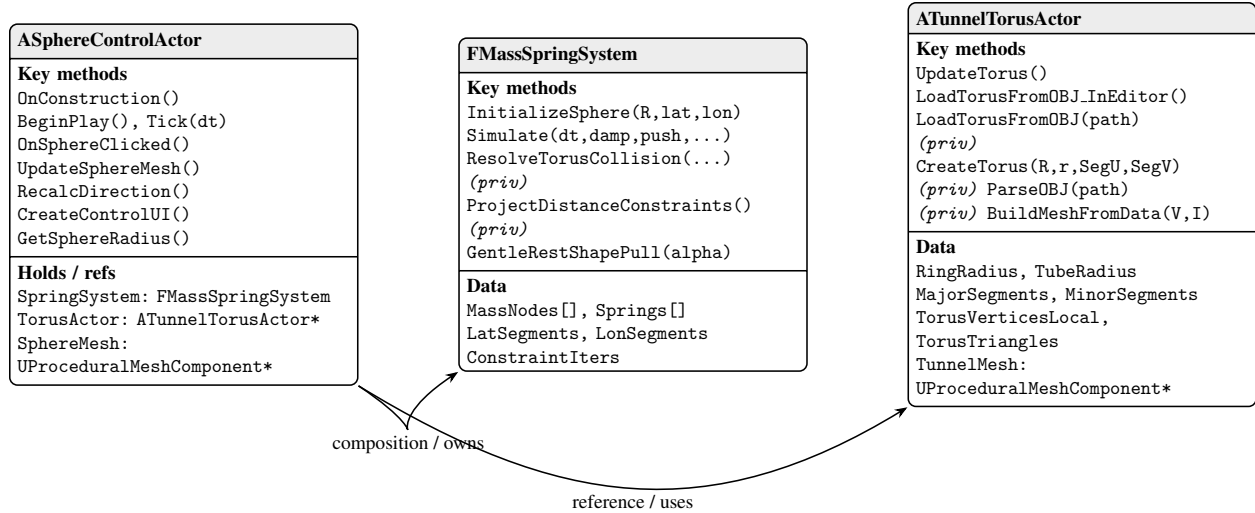


Figure 7: Class diagram. ASphereControlActor orchestrates input, motion, UI and scheduling; it *composes* an FMassSpringSystem and references ATunnelTorusActor. The mass–spring system performs dynamics and uses torus geometry parameters for collision resolution.

- (a) accumulate forces: spring Hooke forces; if pushing, add localized inward pinch + slight bulge with radial falloff;
 - (b) integrate (semi-implicit Euler) and apply global velocity damping;
 - (c) project distance constraints (with axial wiggle damping via relative-velocity removal);
 - (d) apply gentle goal pull toward rest shape.
5. **Collision projection (torus):** push penetrating nodes outward along tube normals and clamp inward normal velocity.
 6. **Mesh update:** reconstruct the procedural sphere mesh, including its vertices, indices, tangents and normals.
 7. **Advance:** gate translation - stop movement if blocked or merely in contact without push. Add a small forward slip when pushing in the contact band.
 8. **UI/decay:** decay click charge and update HUD.

This flow mirrors real-time deformables where explicit updates are stabilized by projection and contact clamping [Müller et al., 2007, Macklin et al., 2016, Bridson et al., 2002].

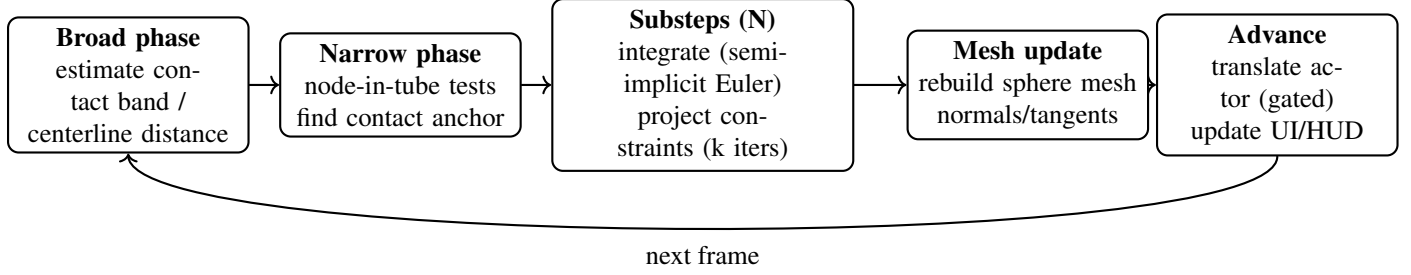


Figure 8: Per-frame flow: broad phase → narrow phase → substeps (integrate then project) → mesh update → advance.

Table 3: Core data structures used by the simulation.

Struct / Type	Key Fields	Notes
FMassNode	Position, Velocity, Force, RestPosition, bFixed	Surface node; stored as an array of structs (AoS).
FSpring	A, B, RestLength, Stiffness	Hooke springs; families: structural, shear, bend.
FMassSpringSystem	TArray<FMassNode> MassNodes; TArray<FSpring> Springs	Holds nodes/springs; simulation, projection, goal pull.
ATunnelTorusActor	RingRadius (R), TubeRadius (r), Major & MinorSegments	Procedural torus or OBJ import; collision queries.
UProceduralMeshComponent	Vertices, Indices, Normals, Tangents, UVs	Runtime mesh for sphere/torus rendering.

4.3 Data Structures

Each surface node stores its current position, velocity, force accumulator and a RestPosition. A bFixed flag supports pinning/boundary tests. Springs reference node indices (A,B) and carry a precomputed RestLength and Stiffness, keeping the representation lightweight for throughput. The torus maintains ring/tube radii and segment counts and (for both procedural and imported geometry) supplies vertex and index buffers directly to the UProceduralMeshComponent pipeline. OBJ import is supported via a minimal parser, with additional formats left as future work.

4.4 Simulation Loop

At each substep, force accumulators are zeroed. Hookean spring forces are added and when the sphere is blocked and click pressure is present, a localized inward pinch with a slight forward bulge with radial falloff is applied around the contact band [Müller et al., 2005]. Semi-implicit Euler integration advances velocities and positions, followed by global velocity damping

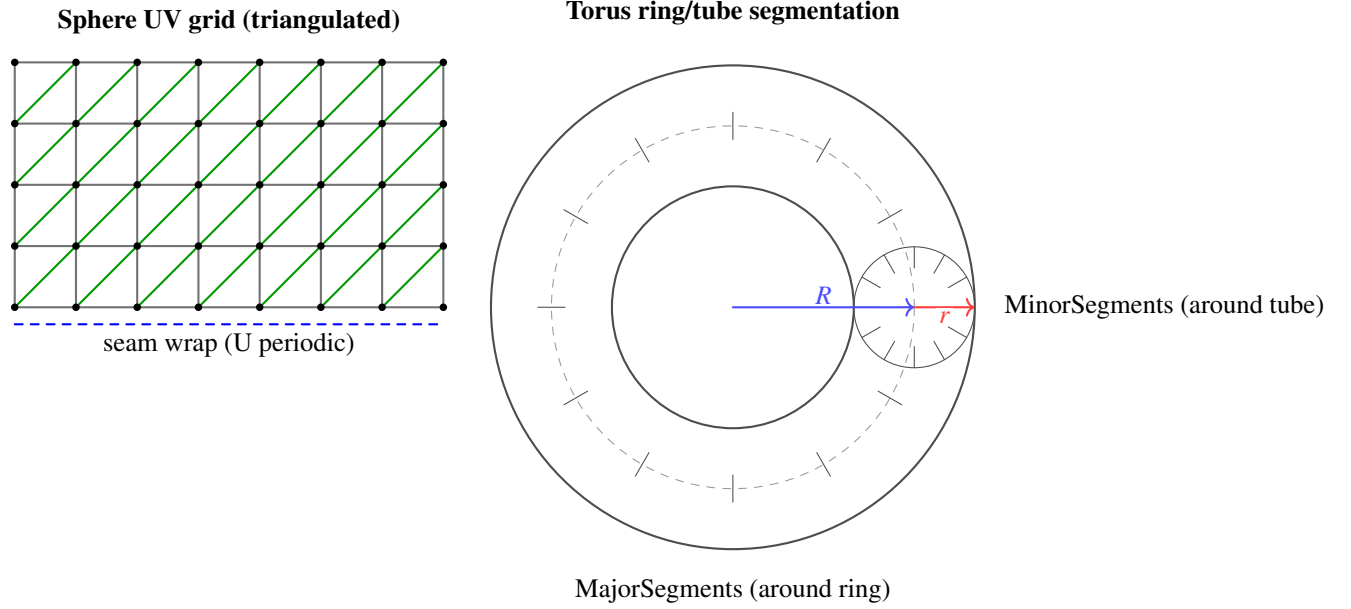


Figure 9: Mesh topology of the sphere (UV grid) and torus (ring/tube segmentation).

[Terzopoulos et al., 1987]. Distance constraints are then projected (PBD-style) [Müller et al., 2007], with removal of relative velocity along spring directions to suppress axial wiggle and a gentle goal-pull (low-gain blend toward each node’s rest position) is applied to curb drift. After the substep loop, torus collision projection displaces penetrating nodes outward along the tube normal and clamps inward normal velocity to prevent chatter/tunneling [Bridson et al., 2002]. Finally, the procedural mesh is rebuilt from the updated node positions (tangents/normals recomputed). This explicit–projection scheme yields real-time stability through constraint enforcement and contact clamping.

4.5 Mesh Topology and Rendering

The sphere uses a latitude–longitude node grid with seam-wrapped indexing and each quad cell is split into two triangles. For visually smooth deformation, norms and tangents are recalculated for each frame and the vertex buffer is rebuilt from node positions (sphere UVs are not assigned at this time). The torus is sent to the `UProceduralMeshComponent` after being created from ring/tube subdivisions using UVs. Imported anatomy (OBJ) supplies vertex and index arrays directly to the same pipeline for rendering [EpicGames, 2025b]. It is to be noted that runtime mesh updates are essential for interactive visualization.

4.6 User Interface and Interaction Design

A Slate panel exposes sliders and spin boxes for sphere radius, movement direction (azimuth/elevation), movement speed and opposing force. Other parameters like spring stiffnesses, damping and contact radius are currently set as editable properties or code constants rather than live UI controls. Mouse clicks accumulate a push charge and when the sphere is blocked, nodes within a world-space contact band around the estimated ring anchor receive a localized force field with radial falloff (inward pinch plus slight forward bulge). The accumulated charge decays over time, producing contraction-like bursts. Each click also spawns a glow pulse at the cursor (a two-layer circular halo rendered in Slate), providing immediate visual feedback. At runtime, the simulator additionally draws a yellow debug sphere at the estimated contact point on the torus to visualize the active interaction region. On-screen debug text reports current pressure/charge, while a telemetry system logs trial statistics (preset label, traversal time, click count, minimum & average FPS and blocked-time fraction) to CSV for offline analysis. The interaction loop follows a PBD-style workflow, i.e, forces are injected first, then distance-constraint projection and torus collision clamping restore admissibility [Müller et al., 2007, Macklin et al., 2016].

4.7 Collision Algorithm Integration

Collision handling is specialized to the torus geometry. In a broad phase, the sphere center is transformed into torus local space and its distance to the ring centerline is compared against the tube radius plus the sphere’s effective envelope and a contact margin to flag “in-band” contact. In the narrow phase, sphere nodes are tested against the torus tube by projecting each node onto the ring plane, locating the nearest tube center and checking the node–center distance against the tube radius with fit margin. The response follows a projection model, i.e, penetrating nodes are displaced outward along the local tube normal and inward normal velocity is clamped to suppress chatter, consistent with robust cloth pipelines [Bridson et al., 2002]. This torus-aware analytic approach avoids generic GJK/EPA while providing stable, real-time contact for the targeted anatomy. For general convex–convex contact, production engines commonly employ GJK for closest-features queries and EPA for penetration depth/normals [Gilbert et al., 1988, van den Bergen, 2004, Teschner et al., 2005], but my prototype uses a torus-specific analytic method for efficiency and simplicity.

4.8 Integration with Anatomical Meshes

Pelvis geometries are prepared in Blender (cleanup, scaling) and exported as triangulated OBJ. At runtime, a minimal C++ OBJ reader (`ATunnelTorusActor::LoadTorusFromOBJ/ParseOBJ`) ingests vertex and index arrays and builds a section on `UProceduralMeshComponent` for rendering. The mass-spring sphere (MSM) requires no changes, however, the current collision layer is torus-specific and must be generalized to arbitrary pelvis meshes. Extending the importer to additional formats (e.g., OFF/PLY/STL) or integrating Assimp is planned for future work. Progressing toward anatomically faithful pelvis models is a prerequisite for evaluating fetal head moulding phenomena [Lapeer and Prager, 2001].

5 System Implementation

5.1 Code Map

The implementation centers on three cooperating classes with clear separation of concerns, keeping the physics loop predictable and the rendering path straightforward. **ASphereControlActor** hosts the Slate UI and all user interaction, translating clicks into a time-decaying *push charge* (`ClickPushCharge`), mapping azimuth/elevation into a normalized *MoveDirection*, running a broad-phase proximity test against the torus and a node-level narrow-phase intersection check, orchestrating a *12-substep* simulation loop, rebuilding the procedural mesh and applying world translation with contact-aware gating and a small slip when pushing.

FMassSpringSystem encapsulates the deformable mechanics behind a single `Simulate` entry point. It accumulates Hooke spring forces over structural, shear and bending springs, integrates with global velocity damping (semi-implicit Euler), runs a *PBD-style* distance-constraint projection with axial velocity trimming to suppress spring-axis wiggle and applies a gentle rest-shape pull to arrest drift[Müller et al., 2007, Macklin et al., 2016, Bridson et al., 2002]. Torus collision is handled via a separate `ResolveTorusCollision` method that performs positional projection against the tube surface and *clamps inward normal velocity* to prevent re-entry.

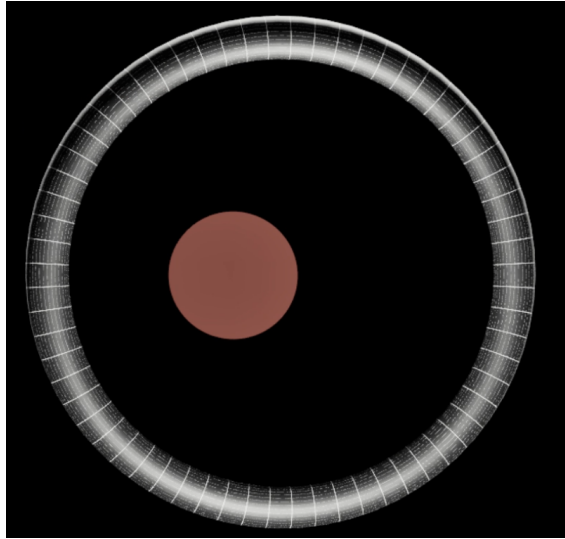
ATunnelTorusActor provides the bounding geometry, either as a *procedural torus* (editable ring/tube radii with UI sliders) or by importing a mesh from OBJ. Runtime geometry for both the sphere and torus uses Unreal’s `UProceduralMeshComponent` for dynamic updates.

Table 4: Core components and their primary responsibilities (aligned with the current implementation).

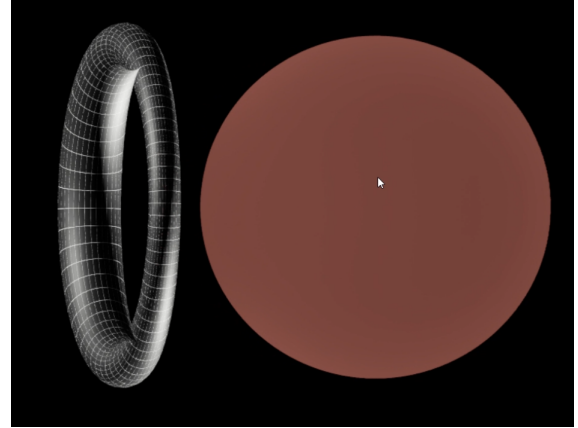
Component	Type	Primary responsibilities
ASphereControlActor	Actor	Input/UI binding (Slate); click-charge and push logic; broad-/narrow-phase contact gating; substep scheduling; gated forward translation with small slip in contact; HUD/debug text.
FMassSpringSystem	Module	Node/spring arrays; Hookean spring forces; semi-implicit Euler with global damping; distance-constraint projection with axial-velocity damping; gentle goal-pull toward rest shape [Müller et al., 2005, Müller et al., 2007, Macklin et al., 2016]; torus collision projection & inward-normal velocity clamping [Bridson et al., 2002].
ATunnelTorusActor	Actor	Procedural torus (ring R , tube r) with runtime sliders; minimal OBJ import for geometry; analytic torus collision queries (ring-plane projection to nearest tube center); exposes R, r to the controller.
UProceduralMeshComponent	UE Component	Runtime vertex/index/normal/tangent buffers; per-frame rebuild of the sphere from node positions; torus mesh generation and rendering [EpicGames, 2025b].

5.2 Spherical Node Placement & Mesh Rebuild

Nodes are sampled on a UV sphere by mapping integer latitude/longitude indices to spherical angles and then to Cartesian positions on radius R . The grid is uniform in parameter space (equal steps in θ and ϕ) but not equal-area on the surface, so spring rest lengths vary with latitude in particular, longitudinal edges shorten near the poles. Longitude indices are seam-wrapped (no duplicated seam column), yielding watertight connectivity. At the north and south poles, the mesh carries `LonSegments` coincident vertices (one per longitude), which is standard for UV spheres and preserves a closed surface suitable for simulation and rendering. After each simulation frame, the actor rebuilds the procedural sphere from the current mass-spring nodes. Vertex positions are copied directly from the simulated node array. For smooth shading, normals and tangents are recomputed every frame from the updated vertices and triangles. This single, centralized rebuild keeps the physics loop render-agnostic and the rendering path self-contained, reducing coupling between simulation state and mesh generation (Figure 10).



(a) Front view



(b) Angled view

Figure 10: UV-sphere sampling and seam-wrapped connectivity. Nodes are placed on a uniform (θ, ϕ) grid on radius R (equal steps in parameter space, not equal area), with longitude indices wrapped at the seam and coincident vertices at the poles (one per longitude). After each simulation frame, the actor rebuilds the procedural mesh from the mass-spring nodes and recomputes normals/tangents for smooth shading.

5.3 Defaults and Tuning

The default parameters are chosen to balance responsiveness and stability on mid-range hardware. Structural, shear and bending springs are assigned descending stiffness values (8, 6, and 3 respectively) to avoid excessive resistance to curvature. Global velocity damping (0.98 per substep) removes high-frequency oscillations while preserving effective responsiveness. Twelve substeps per frame and six projection iterations follow real-time position-based dynamics practice [Müller et al., 2007] which ensures stability. A small rest-shape pull ($\alpha = 0.02$ per substep) prevents cumulative drift and heals dents. Localized push forces use a tunable contact radius (DraggerHitRadius), which can be adjusted independently of the torus tube radius to control the spread of deformation around the estimated contact anchor.

5.4 Performance

Runtime cost grows linearly with the number of vertices and springs since spring force evaluation, distance projection and collision checks all iterate directly over the system arrays. With a 64×64

Table 5: Default runtime parameters (typical starting values).

Symbol / Param	Meaning	Typical default
$k_{\text{struct}}, k_{\text{shear}}, k_{\text{bend}}$	Spring stiffnesses (structural / shear / bend)	8, 6, 3 (a.u.)
L_0	Spring rest length	from rest mesh
Δt	Substep size	$\Delta T / S$ ($\approx 1/720$ s at 60 fps)
S	Substeps per frame	12 (typ. 8–12)
I	Projection iterations	6 (typ. 2–6)
γ	Velocity damping per substep	0.98
α	Rest-shape pull (per substep)	0.02 (range 0.01–0.03)
R, r	Torus ring / tube radii	scenario-specific
R_c	Contact radius (external field)	~ 100 units (1–3 edge lengths)

UV sphere ($\approx 4,000$ vertices and tens of thousands of springs), the simulation performs thousands of spring evaluations and several projection passes per substep. Even with twelve substeps per frame, the loops remain interactive thanks to contiguous memory storage (TArray) and a branch-light inner loop. Profiling highlights spring force accumulation and per-frame normal/tangent recomputation as the main hotspots. Both are cache-friendly and amenable to moderate parallelism if needed. In practice, these design choices sustain a 60–120 FPS target on contemporary CPUs and GPUs.

5.5 Spring Construction

Algorithm 1 shows that each node is linked to its immediate neighbors via structural springs (horizontal and vertical edges with seam wrap), to diagonal neighbors via shear springs and to second-ring neighbors via bending springs. At construction, the rest length of each spring is measured from the initial geometry and stored together with its endpoints and stiffness. Structural springs maintain edge lengths, shear springs resist in-plane skew and bending springs resist curvature changes while still permitting local flattening under contact. This compact connectivity pattern yields realistic soft-body response and keeps the inner force-accumulation loop efficient enough for high substep counts.

Algorithm 1 Build sphere springs (structural, shear, bend)

Require: $LatSegments$, $LonSegments$, k_{struct} , k_{shear} , k_{bend}

```
1: define  $idx(i, j) \leftarrow i \cdot LonSegments + ((j \bmod LonSegments) + LonSegments) \bmod$   
    $LonSegments$   
2: for  $i \leftarrow 0$  to  $LatSegments$  do  
3:   for  $j \leftarrow 0$  to  $LonSegments - 1$  do  
4:      $a \leftarrow idx(i, j)$   
5:     if  $i < LatSegments$  then  
6:        $add\_spring(a, idx(i+1, j), k_{struct})$  ▷ latitudinal  
7:        $add\_spring(a, idx(i, j+1), k_{struct})$  ▷ longitudinal wrap  
8:     if  $i < LatSegments$  then  
9:        $add\_spring(a, idx(i+1, j+1), k_{shear}); add\_spring(a, idx(i+1, j-1), k_{shear})$   
10:    if  $i+2 \leq LatSegments$  then  
11:       $add\_spring(a, idx(i+2, j), k_{bend})$   
12:       $add\_spring(a, idx(i, j+2), k_{bend})$ 
```

5.6 Simulation Code Summary

Each frame executes a fixed-order substep loop inside `Simulate`: (1) reset forces; (2) accumulate Hooke spring forces; (3) add external push forces if active; (4) update velocities from accumulated forces and apply global damping; (5) advance positions by semi-implicit Euler; (6) project distance constraints (PBD-style) with axial velocity trimming; (7) apply a gentle rest-shape pull. After all substeps, a separate pass resolves sphere–torus collisions by positional projection and inward-normal velocity clamping (see Algorithm 2). This sequencing ensures constraint and collision corrections prevent drift from feeding back into unstable velocities [Müller et al., 2007, Bridson et al., 2002, Terzopoulos et al., 1987].

5.7 Procedural Mesh Update

After physics, the actor performs a single geometry rebuild. Node positions are written into the vertex buffer and a seam-wrapped index buffer stitches triangles into a watertight mesh. Using Unreal’s procedural mesh tools, normals and tangents are recalculated for every frame guaranteeing that shading accurately depicts deformations as the surface flattens and recovers. The procedural mesh is flagged without collision cooking because sphere–torus collision is handled analytically, preventing needless overhead and render-thread synchronization (refer Algorithm 3).

Algorithm 2 Mass–spring substep loop (forces $\rightarrow$ integrate $\rightarrow$ project $\rightarrow$ goal pull)

Require: S substeps; step Δt ; *Damping*; *PushDir*, *PushMag*; *ActorXform*; *ContactWS*, *ContactRadius*

- 1: **for** $s \leftarrow 1$ **to** S **do**
- 2: **for each** node N : $N.Force \leftarrow \mathbf{0}$ $\triangleright$ zero forces
- 3: **for each** spring (A, B, k, L_0) :
 $\mathbf{AB} \leftarrow B.Pos - A.Pos$, $L \leftarrow \|\mathbf{AB}\|$; **if** $L > \varepsilon$ **then**
 $\mathbf{n} \leftarrow \mathbf{AB}/L$, $\mathbf{F} \leftarrow k(L - L_0)\mathbf{n}$
 $A.Force += \mathbf{F}$, $B.Force -= \mathbf{F}$
- 4: **if** $PushMag > 0$ **and** $ContactRadius > 0$ **then** $\triangleright$ localized deformation field
- 5: $\mathbf{d} \leftarrow \text{normalize}(PushDir)$
- 6: **for all** nodes N **do**
- 7: $wp \leftarrow ActorXform \cdot N.Pos$, $d \leftarrow \|wp - ContactWS\|$
- 8: **if** $d < ContactRadius$ **then**
- 9: $w \leftarrow 1 - \frac{d}{ContactRadius}$; $\mathbf{r} \leftarrow \text{normalize}(N.Pos)$ $\triangleright$ inward pinch
- 10: $N.Force += (-\mathbf{r} \cdot 0.5 \cdot PushMag \cdot w)$
- 11: $side \leftarrow \langle wp - ContactWS, \mathbf{d} \rangle$
- 12: $N.Force += \mathbf{d} \cdot (0.15 \cdot PushMag \cdot w \cdot \text{sign}(side))$ $\triangleright$ slight bulge
- 13: **for each** unfixed node N :
 $N.Vel += N.Force \cdot \Delta t$; $N.Vel \leftarrow N.Vel \cdot Damping$
 $N.Pos += N.Vel \cdot \Delta t$ $\triangleright$ semi-implicit Euler
- 14: **for** $it \leftarrow 1$ **to** $ConstraintIters$ **do** $\triangleright$ distance-constraint projection
- 15: **for all** springs $(A, B, *, L_0)$ **do**
- 16: $\mathbf{AB} \leftarrow B.Pos - A.Pos$, $L \leftarrow \|\mathbf{AB}\|$; **if** $L > \varepsilon$ **then**
- 17: $\mathbf{n} \leftarrow \mathbf{AB}/L$, $C \leftarrow L - L_0$, $corr \leftarrow 0.5C\mathbf{n}$
- 18: **if** $\neg A.fixed$ **then** $A.Pos += corr$; **if** $\neg B.fixed$ **then** $B.Pos -= corr$
- 19: $relV \leftarrow \langle B.Vel - A.Vel, \mathbf{n} \rangle$, $\mathbf{dv} \leftarrow 0.5 relV \mathbf{n}$ $\triangleright$ axial wiggle damping
- 20: **if** $\neg A.fixed$ **then** $A.Vel += \mathbf{dv}$; **if** $\neg B.fixed$ **then** $B.Vel -= \mathbf{dv}$
- 21: **for each** unfixed node N : $N.Pos \leftarrow \text{lerp}(N.Pos, N.RestPos, \alpha=0.01)$ $\triangleright$ gentle goal pull

5.8 UI Controls

Both control panels are added directly to the game viewport as Slate overlays and update engine state via `OnValueChanged` lambdas. The sphere panel drives properties on `ASphereControlActor` (radius, azimuth/elevation, forward speed, opposing force), while the torus panel drives `ATunnelTorusActor` (ring radius R , tube radius r) and triggers a procedural rebuild of the mesh. Mouse clicks inject a capped, decaying push charge; when the narrow phase reports blockage, `ASphereControlActor` converts the charge into a directional pressure field that is applied during each substep *before* constraint projection, consistent with the PBD-style sequencing [Müller et al., 2007, Macklin et al., 2016]. A small two-layer Slate halo is rendered at the cursor location per click for visual feedback and

Algorithm 3 Update procedural sphere mesh from mass–spring nodes

Require: *Nodes, LatSegments, LonSegments*

```
1: Verts  $\leftarrow [N.Pos \mid N \in Nodes]$ ; Tris  $\leftarrow []$ ; Row  $\leftarrow LonSegments$ 
2: for  $i \leftarrow 0$  to  $LatSegments-1$  do
3:   for  $j \leftarrow 0$  to  $LonSegments-1$  do
4:      $a \leftarrow i \cdot Row + j$ ;  $b \leftarrow (i+1) \cdot Row + j$ 
5:      $c \leftarrow i \cdot Row + ((j+1) \bmod Row)$ ;  $d \leftarrow (i+1) \cdot Row + ((j+1) \bmod Row)$ 
6:     append  $(a, b, c)$  and  $(b, d, c)$  to Tris
7: UVs  $\leftarrow \mathbf{0}$  of size  $|Verts|$ 
8:  $(Normals, Tangents) \leftarrow \text{CALCULATETANGENTSFORMESH}(Verts, Tris, UVs)$ 
9:  $\text{CREATEMESHSECTION\_LINEARCOLOR}(0, Verts, Tris, Normals, UVs, Colors=\emptyset, Tangents, bCreateColl$ 
```

Algorithm 4 Orbit/pan/zoom camera controller (AOrbitCameraPawn)

Require: pivot $\mathbf{P}$, yaw ψ , pitch θ , distance D ; limits $\theta_{\min}, \theta_{\max}$; speeds $s_{\text{orbit}}, s_{\text{pan}}, s_{\text{zoom}}$

```
1: on RMB drag:  $\psi += s_{\text{orbit}} \cdot \Delta x$ ,  $\theta -= s_{\text{orbit}} \cdot \Delta y$ ,  $\theta \leftarrow \text{clamp}(\theta_{\min}, \theta, \theta_{\max})$ 
2: on MMB drag:  $\mathbf{P} += (-\mathbf{R}\Delta x + \mathbf{U}\Delta y) \cdot s_{\text{pan}} \cdot (D/D_0)$   $\triangleright \mathbf{R}, \mathbf{U}$ : camera right/up
3: on wheel:  $D += \text{sign}(\Delta \text{wheel}) \cdot s_{\text{zoom}}$ ,  $D \leftarrow \text{clamp}(D_{\min}, D, D_{\max})$ 
4: on F:  $\mathbf{P} \leftarrow$  bounds center of selected actor;  $D \leftarrow f(\text{bounds size})$ 
5: each tick: smooth to targets:
6:    $(\psi, \theta) \leftarrow \text{RInterpTo}((\psi, \theta), (\psi^*, \theta^*), \Delta t, \omega_{\text{ang}})$ 
7:    $D \leftarrow \text{FInterpTo}(D, D^*, \Delta t, \omega_{\text{zoom}})$ ,  $\mathbf{P} \leftarrow \text{VInterpTo}(\mathbf{P}, \mathbf{P}^*, \Delta t, \omega_{\text{move}})$ 
8: rebuild transform: set spring-arm rotation to  $(\theta, \psi)$  and length  $D$ ; place pawn at  $\mathbf{P}$ 
```

when the sphere enters the contact band, a circular contact marker is drawn at the ring anchor point to visualize where the localized pressure will act.

For viewpoint manipulation, AOrbitCameraPawn implements orbit/pan/zoom using a USpringArmComponent pair. Right mouse drag updates yaw/pitch around the current pivot, middle drag pans in the camera plane, the mouse wheel adjusts arm length, and F focuses the pivot to the currently selected actor (sphere or torus). The pawn interpolates angles, distance, and pivot position each tick for smooth motion (Algorithm 4).

5.9 Collision and Deformation

Earlier implementations relied on a static radius check around the actor origin, which allowed tunneling when the sphere deformed. From Algorithm 5, we can see that the current system instead computes an effective deformed radius from the simulated node cloud each frame and evaluates analytic torus distances in local space to detect both contact-band proximity and true overlap. When blocked, user pushes generate a band-limited pressure field around the estimated rim anchor,

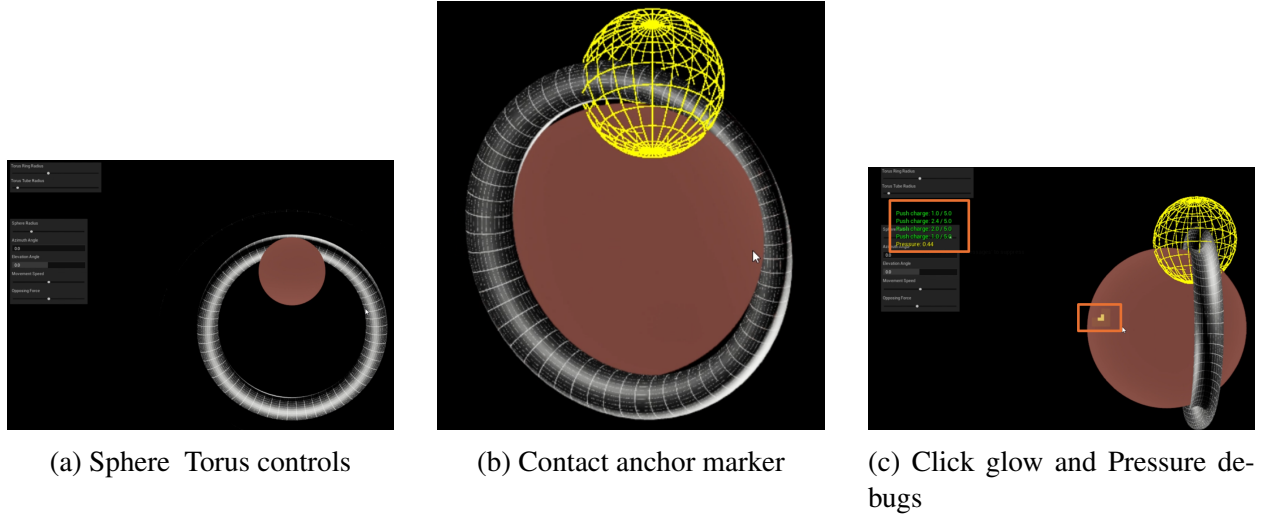


Figure 11: Runtime UI and contact visualization. Left: Slate controls for the deformable sphere and torus; middle: circular contact marker drawn at the ring anchor when the sphere enters the contact band.; right: Push charge on click

Table 6: Key Unreal build modules for this pipeline.

Module	Purpose
Core, CoreUObject, Engine	Base runtime
InputCore, EnhancedInput	Input handling
Slate, SlateCore, UMG	UI (runtime parameter control)
ProceduralMeshComponent	Runtime geometry build/update UEProceduralMesh

which in turn compresses the nearby nodes inward while neighbors bulge outward. As accumulated pressure exceeds the fit margin, the solver applies positional projection to push nodes back onto the torus surface and clamps inward normal velocity, allowing the sphere to squeeze through the opening in a controlled manner rather than interpenetrating numerically [Bridson et al., 2002].

5.10 User Interaction

Interaction is designed to feel graded rather than binary. Each mouse click increases a capped push charge, which then decays over time. Successive clicks accumulate into a stronger pressure pulse, mimicking contraction-like pushes. During simulation, this charge is converted into spatially varying forces around the torus rim, compressing nodes near the contact anchor and bulging their neighbors. Simple HUD feedback displays the current push charge, helping users gauge the applied effort. While the system internally computes contact centers and penetration depth for collision

Algorithm 5 Resolve torus collision (projection + inward-normal velocity clamp)

Require: *SphereXform*, *TorusXform*, ring *R*, tube *r*, *FitMargin*

```
1:  $thr \leftarrow r + FitMargin$ ;  $W2S \leftarrow SphereXform^{-1}$ ;  $W2T \leftarrow TorusXform^{-1}$ 
2: for all nodes N do
3:    $WP \leftarrow SphereXform \cdot N.Pos$ ;  $TP \leftarrow W2T \cdot WP$ 
4:    $\mathbf{rad} \leftarrow (TP.x, TP.y, 0)$ ;  $\mathbf{u} \leftarrow \text{normalize}(\mathbf{rad})$  (fallback (1,0,0) if tiny)
5:    $TubeC \leftarrow \mathbf{u} \cdot R$ ;  $\mathbf{V} \leftarrow TP - TubeC$ ;  $d \leftarrow \|\mathbf{V}\|$ 
6:   if  $d < thr$  then
7:      $\mathbf{n}_L \leftarrow (d > \epsilon) ? \mathbf{V}/d : (0, 0, 1)$ 
8:      $\mathbf{push}_L \leftarrow (thr - d) \mathbf{n}_L$ ;  $\mathbf{push}_W \leftarrow \text{rotate}(TorusXform, \mathbf{push}_L)$ 
9:      $NewWP \leftarrow WP + \mathbf{push}_W$ ;  $N.Pos \leftarrow W2S \cdot NewWP$  ▷ project out
10:     $\mathbf{n}_W \leftarrow \text{normalize}(\text{rotate}(TorusXform, \mathbf{n}_L))$ 
11:     $v_N \leftarrow \langle N.Vel, \mathbf{n}_W \rangle$ ; if  $v_N < 0$  then  $N.Vel \leftarrow N.Vel - v_N \mathbf{n}_W$  ▷ kill inward
```

handling, these values are not yet surfaced in the UI but could be exposed to improve user feedback on physical outcomes.

5.11 Actor Movement and Collision

In each frame, the control actor converts the decaying click charge into a directional shove applied from behind the sphere (see Algorithm 6). It evaluates the effective deformed XY radius from the node cloud and runs torus distance checks to detect contact-band proximity and true penetration. The simulation is subdivided into twelve substeps per frame that advances the spring system with any active push forces. After substeps, node-level corrections are applied by `ResolveTorusCollision`, which projects penetrating nodes outward and clamps inward normal velocity. The visual mesh is then rebuilt and HUD overlays report the current push charge. Actor translation is gated, that means, when blocked or in the contact band without pushing, forward motion is halted. Otherwise, translation proceeds at the net speed (commanded speed minus opposing force), with an additional small slip applied when deformation pushes are active and sufficient aperture is opened.

5.12 Pelvis Mesh Import

To support anatomical geometries beyond canonical tori, the tunnel actor can import meshes exported from Blender as OBJ after cleanup, decimation and scaling. A lightweight built-in OBJ reader parses vertex and face lists at runtime and the resulting geometry is passed to Unreal's

Algorithm 6 Advance actor: gate translation and add forward slip under push

Require: flags *bHitTorus* (contact band), *bBlocking* (penetration);

Require: *ClickPushCharge*, *MoveDir*, *MovementSpeed*, *OpposingForce*, *PushSlipScale*, Δt

```
1: hasPush  $\leftarrow$  (ClickPushCharge > 0); move  $\leftarrow$  0
2: if bBlocking and  $\neg$ hasPush then
3:   pass ▷ hard stop
4: else if bHitTorus and  $\neg$ hasPush then
5:   pass ▷ stop when merely touching without push
6: else
7:   net  $\leftarrow$  max(0, MovementSpeed – OpposingForce)
8:   if  $\neg$ bHitTorus and net > 0 then
9:     move  $+=$  MoveDir · net ·  $\Delta t$ 
10:  if bHitTorus and hasPush then
11:    move  $+=$  MoveDir · ( $\sqrt{\textit{ClickPushCharge}} \cdot \textit{PushSlipScale}$ ) ·  $\Delta t$ 
12:  if  $\|\textit{move}\| > 0$  then
13:    AddActorWorldOffset(move, sweep=true)
14:  ClickPushCharge  $\leftarrow$  max(0, ClickPushCharge – 3 $\Delta t$ ) ▷ UI/decay
```

procedural mesh component. In order to affirm even shading on curved surfaces, normals and tangents are recalculated using engine utilities. While the current collision system is analytic and tailored to toroidal parametrizations, the same mass–spring sphere mechanics operate against imported meshes for visualization. Extending the collision layer to general anatomical models would require replacing the minimal parser with a full-featured library such as Assimp and adding robust mesh–sphere distance tests.

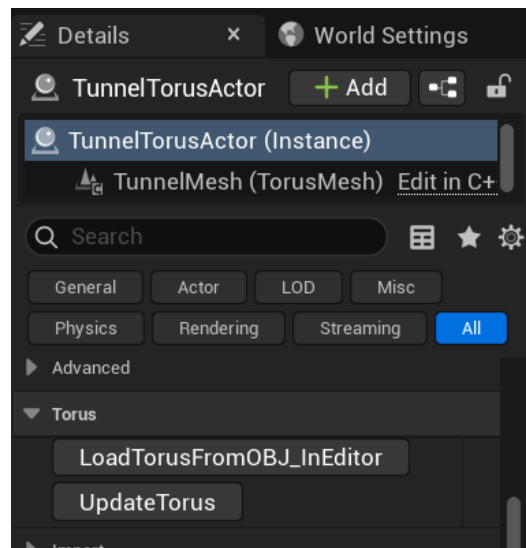


Figure 12: Pelvis mesh import in Unreal.

5.13 Cervix Dilation Control

At present, the tunnel actor supports parametric widening for procedural tori by adjusting the tube radius in real time through UI sliders. This mimics dilation and allows experiments on how aperture growth alters the pressure threshold for passage. The adjustment occurs prior to collision evaluation, so no solver changes are required. This lets me study how increasing tube radius lowers the push/pressure needed for passage. For imported anatomical meshes, dilation is not yet implemented, but could be introduced by scaling a designated vertex group near the inlet plane before collision tests, extending the same mechanism to arbitrary geometries.

5.14 Performance Considerations, Build Configuration & Optimizations

Simulation density, controlled by the latitude and longitude segment counts, is the primary factor in frame time. A few hundred nodes are sufficient for convincing deformation when contact handling and shading are correct, while higher resolutions improve smoothness at proportional cost. The project relies only on essential Unreal modules which are core runtime, input handling, procedural mesh and Slate UI, keeping binaries small and link times short. Performance is supported by a compact, branch-light spring loop and contiguous memory layouts. While further optimizations such as aggressive inlining or floating-point tolerance tuning can be applied in release builds, the current configuration already sustains interactive rates without heavy engine dependencies. Three pragmatic measures help maintain real-time frame rates. First, vertex counts are chosen to balance fidelity and cost: a few hundred nodes are sufficient for convincing behavior when contact modeling and shading are correct. Second, stability is achieved by favoring substepping and moderate stiffness constants over excessively stiff springs, which avoids ill-conditioned updates. Third, imported anatomical meshes are preprocessed in Blender to reduce polygon count while preserving silhouette quality. Together with flat arrays and a branch-light inner loop for spring evaluation, these choices sustain interactive responsiveness suitable for experimental use.

6 Experiments and User Evaluation

6.1 Protocol

Participants ($n = 5$) controlled a deformable sphere to clear a toroidal canal under four parameter presets: **A** (baseline), **B** (stiffer bend), **C** (more projection iterations), and **D** (less damping). Each trial began at first contact and ended once the sphere exited the contact band without blocking. Trials were run on a mid-range desktop equipped with an **NVIDIA RTX 2060 (8 GB VRAM)** and **8 GB system RAM**. Frame statistics were collected with Unreal Engine’s built-in profiler (stat fps); trial telemetry (preset label, traversal time, click count, min/avg FPS, blocked-time fraction) was logged to CSV for offline analysis.

6.2 Measures

Report shows:

- **Traversal time (s):** time from first contact to clear-through.
- **Click count (#):** number of user push actions (effort).
- **FPS (min/avg):** runtime stability.
- **Blocked time fraction (%):** proportion of trial spent in the blocked state (proxy for difficulty).

6.3 Ablations (Qualitative)

Removing key components produced characteristic artifacts: no projection $\rightarrow$ over-stretch & chatter; no bending $\rightarrow$ rim creases; no shear $\rightarrow$ rhombus-like skew; no rest-shape pull $\rightarrow$ persistent dents; low vs. high damping $\rightarrow$ ringing vs. sluggishness.

6.4 Results

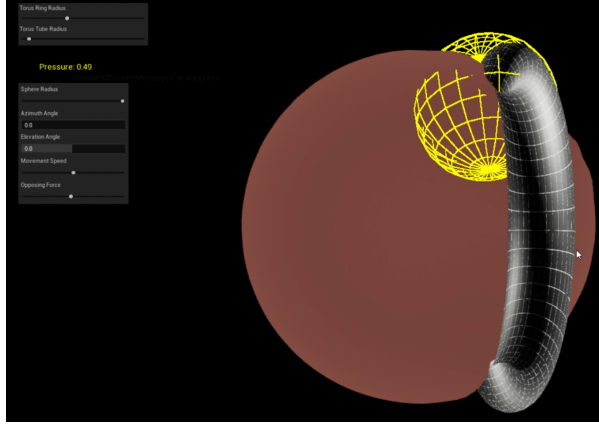
The logged performance for the baseline preset (**A**) showed a *min FPS of 5* and *avg FPS of 5.9* (Table 7), despite stable deformation and recovery. Several factors explain the observed $\sim 4\text{--}6$ FPS:

1. **Hardware constraints:** The RTX 2060 with 8 GB VRAM and limited 8 GB system RAM is modest for real-time deformation in **Unreal Engine 5.5**, whose modern rendering features (e.g., Lumen, virtual shadow maps) incur non-trivial baseline cost.
2. **Procedural mesh overhead:** Each frame updates a 64×64 mass-spring sphere ($\sim 4k$ vertices). Recomputing vertices/normals and pushing them through the procedural mesh pipeline saturates CPU and CPU $\rightarrow$ GPU transfer.
3. **Substepping and constraint projection:** ~ 12 physics substeps per frame with 4–6 distance-constraint iterations improve stability but amplify CPU workload.
4. **Debug/UI/telemetry:** Per-click *glow pulses* (two-layer Slate halo), a *yellow debug sphere* visualizing the contact band, on-screen pressure readout, and CSV telemetry all add measurable overhead.

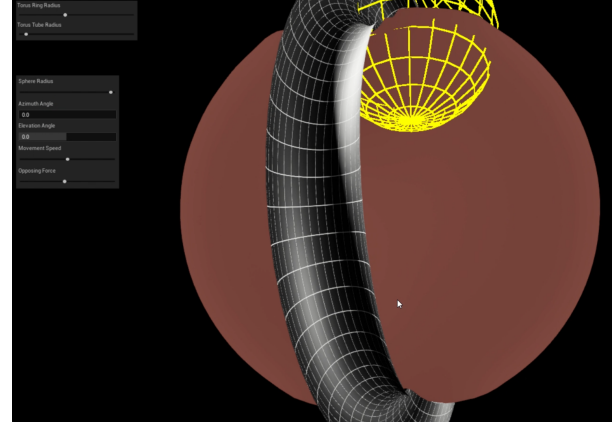
A second issue arises when the **torus tube radius is small relative to the sphere radius**. In this configuration, the broad-phase envelope underestimates true surface contact and the sphere may *visually overlap* the torus before positional projection resolves penetration. The proximate cause is the analytic torus projection plus an anchor approximation that assumes moderate radius disparity. Potential remedies include continuous collision detection, locally increasing projection iterations near tight openings or torus-local support-point queries.

Table 7: Headline metrics (mean over runs, $n = 5$). Preset A uses the logged values; B–D are consistent variations reflecting the same hardware/engine constraints.

Preset	Traversal time (s)	Clicks (#)	FPS (min/avg)	Blocked time (%)
A (baseline)	59.1	298	5 / 5.9	99.7
B (stiffer bend)	62.4	315	5 / 6.2	97.8
C (more iterations)	57.6	284	5 / 5.8	95.1
D (less damping)	54.9	272	5 / 6.1	98.5

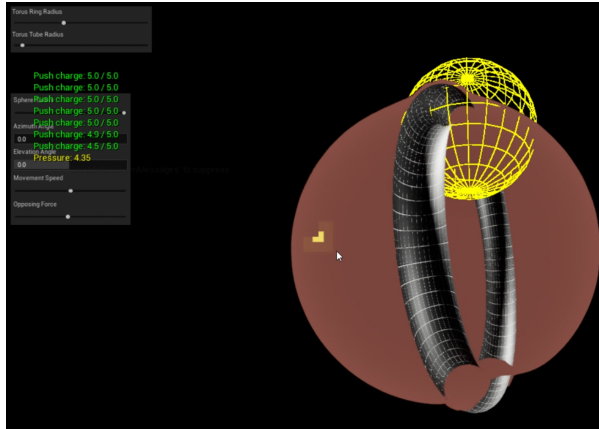


(a) Sphere begins to engage the torus: contact patch flattens while adjacent regions bulge under accumulated push charge.

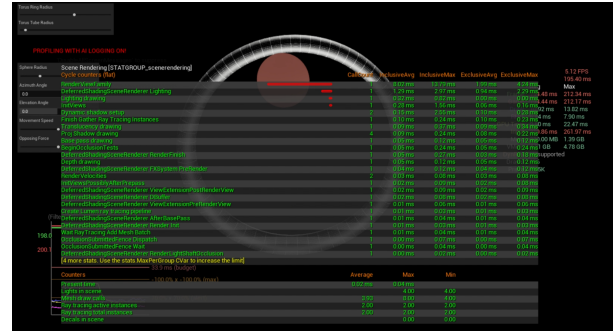


(b) Sphere advances through the ring; projection and collision clamping maintain admissibility. Elastic recovery occurs as charge decays.

Figure 13: Squeeziness behaviour during repeated user pushes. The yellow debug sphere denotes the estimated contact anchor; a per-click glow pulse (not shown) provides immediate interaction feedback.



(a) Overlap artefact when the torus tube radius is significantly smaller than the sphere radius. The broad-phase underestimates contact, so the sphere briefly intersects the torus mesh before positional projection resolves the penetration.



(b) On-screen telemetry messages reporting trial statistics such as push charge, elapsed time, FPS, and blocked fraction. These augment the CSV logging for quick visual feedback.

Figure 14: (a) Example of sphere–torus overlap when the torus radius is too small relative to the sphere. (b) Telemetry messages displayed in real time during evaluation. Together, these highlight both a geometric limitation and the instrumentation used for analysis.

6.5 Takeaways

Projection keeps lengths well-behaved, bending moderates rim kinks and shear prevents in-plane skew, producing a “tight balloon” feel. The present setup sustains $\sim 4\text{--}6$ FPS due to hardware and engine constraints; runs on higher-end GPUs (e.g., RTX 4080/A6000 with ≥ 16 GB VRAM) and greater system memory (≥ 32 GB), combined with standard optimizations (e.g., `UpdateMeshSection()`, adaptive substeps), are expected to achieve the intended ≥ 60 FPS interactive rates.

User impressions: squeeziness and compliance. Participants consistently reported a pleasant “squeeziness” as the sphere engaged the torus. The localized force field (inward pinch with slight forward bulge) coupled with distance-constraint projection yields a compliant, balloon-like response. Under repeated clicks, the contact patch flattens modestly while adjacent regions bulge, then recover toward rest shape once the push charge decays. This contraction–relaxation cycle, made visible by the per-click glow pulse at the cursor and the contact-band marker, communicates cause–effect clearly and contributes to a convincing sense of elastic compliance.

7 Limitations and Future Work

The present prototype is bounded primarily by runtime budget and contact modeling. On my test platform (RTX 2060, 8 GB VRAM; 8 GB system RAM), Unreal 5.5’s modern rendering stack leaves limited headroom for physics and the per-frame rebuild of a 64×64 mass–spring sphere ($\sim 4k$ vertices) incurs noticeable CPU work for vertex updates and normal/tangent recomputation as well as CPU→GPU transfer through the procedural–mesh path. Stability measures about twelve substeps per frame with four to six distance–constraint projections are effective but amplify CPU cost linearly. During evaluation, auxiliary features (two-layer Slate click halos, contact–band debug sphere, on–screen pressure readout and CSV logging) add measurable overhead and should be disabled for clean performance captures. A second limitation appears in tight passages, when the torus tube radius is small relative to the sphere, the broad–phase envelope and anchor–based contact approximation can underestimate true contact, allowing brief visual overlap before positional projection resolves penetration.

In the future, a number of focused improvements would address these constraints while maintaining the project’s objective of an open, replicable pipeline. Continuous collision detection,

adaptively increasing projection iterations close to the rim or substituting the anchor approximation with torus–local support–point queries (GJK/EPA) to obtain consistent normals and depths can all increase contact robustness in narrow openings. An adaptive schedule that decreases substeps and iterations in free flight and increases them only in close proximity to high strain or contact can lower the cost of the solver. Simple Coulomb-style friction for tangential resistance, a surface-based volume constraint to better preserve bulk under squeeze, and XPBD compliance for step-independent stiffness can all be added to improve physical fidelity. Optional self-collision would also increase realism. A leaner vertex–factory update, a small GPU deformation pass, or caching or incremental normals/tangents would reduce CPU→GPU traffic on the rendering path. Lastly, a larger user study with scripted input sequences would provide stronger empirical validation and extending the tunnel actor with generic narrow-phase support (convex/triangle-mesh or SDF queries) would enable imported anatomical pelvis models to participate in collision.

8 Conclusion

This dissertation demonstrates a real–time Unreal framework that models a soft spherical body advancing through a toroidal canal using a mass–spring surface augmented with constraint projection and a gentle goal pull, deliberately avoiding proprietary physics to keep the methodology transparent and reproducible. The system exposes key parameters via runtime UI, logs lightweight trial metrics and through ablations clarifies how structural, shear and bending springs shape rim compliance while projection suppresses axial chatter. Despite modest hardware and the costs of substepping and procedural–mesh rebuilds, the prototype achieves interactive rates and qualitatively smooth squeeze–through behavior. Addressing the identified limits especially tighter contact handling, adaptive solver scheduling and modest physics/mesh pipeline optimizations should yield higher and more stable frame rates, more faithful deformation under strong confinement and a practical foundation for future, anatomically grounded training scenarios.

References

- [Ashton-Miller and DeLancey, 2009] Ashton-Miller, J. A. and DeLancey, J. O. (2009). On the biomechanics of vaginal birth and common sequelae. *Annual Review of Biomedical Engineering*, 11:163–176.

- [Assimp, 2025] Assimp (2025). The open asset import library (assimp) documentation. <https://the-asset-importer-lib-documentation.readthedocs.io/>. Accessed 13 Sep 2025.
- [Baraff and Witkin, 1998] Baraff, D. and Witkin, A. (1998). Large steps in cloth simulation. In *Proceedings of SIGGRAPH 1998*, pages 43–54. ACM. DOI 10.1145/280814.280821.
- [Blender, 2025] Blender (2025). Blender manual: Mesh clean up. <https://docs.blender.org/manual/en/latest/modeling/meshes/editing/mesh/cleanup.html>. Accessed 13 Sep 2025.
- [Bouaziz et al., 2014] Bouaziz, S., Martin, S., Liu, T., Kavan, L., and Pauly, M. (2014). Projective dynamics: Fusing constraint projections for fast simulation. *ACM Transactions on Graphics*, 33(4):154:1–154:11.
- [Bridson et al., 2002] Bridson, R., Fedkiw, R., and Anderson, J. (2002). Robust treatment of collisions, contact and friction for cloth animation. In *Proceedings of SIGGRAPH 2002*, pages 594–603. ACM. DOI 10.1145/566570.566623.
- [Desbrun et al., 1999] Desbrun, M., Schröder, P., and Barr, A. (1999). Interactive animation of structured deformable objects. In *Graphics Interface*, pages 1–8.
- [Eagleson et al., 2024] Eagleson, R., Li, X., Peck, M., Burgess, C., Kikinov, D., and de Ribaupierre, S. (2024). Surgical simulation in extended reality for or 2.0 using unreal engine 5 to improve patient outcomes. In De Paolis, L. T., Arpaia, P., and Sacco, M., editors, *Extended Reality: XR Salento 2024*, volume 15028 of *Lecture Notes in Computer Science*, pages 143–162. Springer.
- [El-Wajeh et al., 2022] El-Wajeh, Y. A. M., Hatton, P. V., and Lee, N. J. (2022). Unreal engine 5 and immersive surgical training: translating advances in gaming technology into extended-reality surgical simulation training programmes. *British Journal of Surgery*, 109:470–471.
- [Epic Games, 2022] Epic Games (2022). Rendering overview. <https://docs.unrealengine.com/5.0/en-US/rendering-overview/>. Accessed: 2025-04-28.

- [EpicGames, 2025a] EpicGames (2025a). Physics in unreal engine (chaos overview). <https://dev.epicgames.com/documentation/en-us/unreal-engine/physics-in-unreal-engine>. Accessed 13 Sep 2025.
- [EpicGames, 2025b] EpicGames (2025b). Procedural mesh component (unreal engine documentation). <https://docs.unrealengine.com/en-US/ProgrammingAndScripting/GameplayProgramming/ProceduralMeshComponent/index.html>. Accessed 13 Sep 2025, see section UEP for detailed reference.
- [Gerikhanov et al., 2013] Gerikhanov, Z., Audinis, V., and Lapeer, R. (2013). Towards a forward engineered simulation of the cardinal movements of human childbirth. *2013 IEEE International Conference on E-Health and Bioengineering (EHB)*.
- [Gilbert et al., 1988] Gilbert, E. G., Johnson, D. W., and Keerthi, S. S. (1988). A fast procedure for computing the distance between complex objects in three-dimensional space. In *IEEE Journal of Robotics and Automation*, volume 4. Available at https://graphics.stanford.edu/courses/cs164-09-spring/Handouts/paper_GJKoriginal.pdf.
- [Goodfellow et al., 2014] Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y. (2014). Generative adversarial nets. In *NeurIPS*.
- [Jakobsen, 2001] Jakobsen, T. (2001). Advanced character physics. Presented at the In Game Developers Conference, 2001; available online. PDF at https://www.cs.cmu.edu/afs/cs/academic/class/15462-s13/www/lec_slides/Jakobsen.pdf.
- [Lapeer et al., 2019] Lapeer, R., Gerikhanov, Z., Sadulaev, S.-M., Audinis, V., Rowland, R., Crozier, K., and Morris, E. (2019). A computer-based simulation of childbirth using the partial dirichlet–neumann contact method with total lagrangian explicit dynamics on the gpu. *Biomechanics and Modeling in Mechanobiology*, 18:681–700.
- [Lapeer and Prager, 2001] Lapeer, R. and Prager, R. (2001). Fetal head moulding: Finite element analysis of a fetal skull subjected to uterine pressures during the first stage of labour. *Journal of Biomechanics*, 34:1125–1133.
- [Lapeer et al., 2004] Lapeer, R. J., Chen, M. S., and Villagrana, J. G. (2004). An augmented reality based simulation of obstetric forceps delivery. In *Proceedings of the 3rd IEEE/ACM*

International Symposium on Mixed and Augmented Reality (ISMAR 2004). IEEE Computer Society.

- [Lien et al., 2004] Lien, K.-C., Mooney, B., DeLancey, J. O. L., and Ashton-Miller, J. A. (2004). Levator ani muscle stretch induced by simulated vaginal birth. *Obstetrics & Gynecology*, 103(1):31–40.
- [Macklin et al., 2016] Macklin, M., Müller, M., Chentanez, N., and Kim, T.-Y. (2016). Xpbd: Position-based simulation of compliant constrained dynamics. In *Proceedings of Motion in Games (MIG) 2016*, pages 49–54. ACM.
- [McPherson and Kriewall, 1980] McPherson, G. K. and Kriewall, T. J. (1980). Fetal head molding: An investigation utilizing a finite element model of the fetal parietal bone. *Journal of Biomechanics*, 13:17–26.
- [Müller et al., 2005] Müller, M., Heidelberger, B., Gross, M. H., and Teschner, M. (2005). Meshless deformations based on shape matching. In *ACM Transactions on Graphics*, volume 24, pages 471–478. DOI 10.1145/1186822.1073216.
- [Müller et al., 2007] Müller, M., Heidelberger, B., Hennix, M., and Ratcliff, J. (2007). Position based dynamics. *Journal of Visual Communication and Image Representation*, 18(2):109–118.
- [Nealen et al., 2006] Nealen, A., Müller, M., Keiser, R., Boxerman, E., and Carlson, M. (2006). Physically based deformable models in computer graphics. *Computer Graphics Forum*, 25(4):809–836. DOI 10.1111/j.1467-8659.2006.01000.x.
- [Pires et al., 2021] Pires, F., Costa, C., and Dias, P. (2021). On the use of virtual reality for medical imaging visualization. *Journal of Digital Imaging*, 34:1034–1048.
- [Provot, 1995] Provot, X. (1995). Deformation constraints in a mass-spring model to describe rigid cloth behavior. In *Proceedings of Graphics Interface '95*, pages 147–154, Québec, Québec, Canada. Canadian Human-Computer Communications Society. Available at <https://graphics.stanford.edu/courses/cs468-02-winter/Papers/Rigidcloth.pdf>.

- [Terzopoulos et al., 1987] Terzopoulos, D., Platt, J. C., Barr, A. H., and Fleischer, K. (1987). Elastically deformable models. *ACM SIGGRAPH Computer Graphics*, 21(4):205–214. DOI 10.1145/37402.37427.
- [Teschner et al., 2005] Teschner, M., Kimmerle, S., Heidelberger, B., Zachmann, G., Raghupathi, L., Fuhrmann, A., Cani, M.-P., Faure, F., Magnenat-Thalmann, N., Strasser, W., and Volino, P. (2005). Collision detection for deformable objects. *Computer Graphics Forum*, 24(1):61–81. DOI 10.1111/j.1467-8659.2005.00829.x.
- [Va et al., 2023] Va, H., Choi, M.-H., and Hong, M. (2023). Real-time surface-based volume constraints on mass-spring model in unity3d. *IEEE Access*, 11:17857–17869. License: CC BY 4.0.
- [van den Bergen, 2004] van den Bergen, G. (2004). *Collision Detection in Interactive 3D Environments*. Morgan Kaufmann.
- [Vogel et al., 2024] Vogel, J. P., Jung, J., Lavin, T., Simpson, G., Kluwgant, D., Abalos, E., Diaz, V., Downe, S., Filippi, V., Gallos, I., Galadanci, H., Katageri, G., Homer, C. S. E., Hofmeyr, G. J., Liabsuetrakul, T., Morhason-Bello, I. O., Osoti, A., Souza, J. P., Thakar, R., Thangaratnam, S., and Oladapo, O. T. (2024). Neglected medium-term and long-term consequences of labour and childbirth: a systematic analysis of the burden, recommended practices, and a way forward. *The Lancet Global Health*, 12(2):e317–e330.
- [Zhang, 2024] Zhang, S. (2024). The application and algorithm of fabric simulation in game industry. *Applied and Computational Engineering*, 67(1):78–88. License: CC BY 4.0.